

Cutting Down the Tower: Single-Exponential Envy-Free Cake Cutting

Qilin Ye
Stanford University
yql@stanford.edu

Yannan Bai
Carnegie Mellon University
byn@cmu.edu

Abstract

Envy-free cake cutting is a central problem in fair division with a striking divide between existence and computation. Classical topology guarantees that envy-free allocations exist, yet finding one efficiently turned out to be much harder, and this problem has resisted decades of work. A well-known result by Aziz and Mackenzie [3] established the existence of a bounded protocol for every n , but its query bound is $n^{n^{n^n}}$. A tighter analysis by Sokolov [30] subsequently reduced this upper bound to $n^{8n^2(1+o(1))}$, the best known prior to this work. In contrast, the general lower bound, due to Procaccia [26], is merely $\Omega(n^2)$.

We close much of this massive gap with a protocol using at most $n^{O(1)}2^n$ queries. At a high level, our protocol repeatedly allocates some cake without creating envy until the remaining problem involves fewer agents. The main difficulty is to ensure that, when we later put these allocations together, we neither assign any cake twice nor create envy. To overcome this difficulty, we develop a new construction using only polynomially many partial allocations, replacing the n^{n^n} partial allocations used in previous work. Overall, our protocol gives the first single-exponential query bound for finding a complete envy-free allocation with arbitrary nonatomic, additive valuations.

Contents

1	Introduction	3
1.1	Previous Work	3
1.2	Our Results and Techniques	3
2	Preliminaries	5
3	The Protocol at a High Level	6
3.1	One Recursive Call	6
4	Proof of the Main Theorem	8
4.1	Basic Facts	8
4.2	A Lemma for Cutting Several Pieces at Once	11
4.3	Pre-processing the Snapshots	12
4.4	Pruning the Snapshots	14
4.5	Constructing the Partial Allocation	15
4.6	Recursion and Query Complexity	22

5	Discussion and Concluding Remarks	23
A	AI Usage Disclosure	27
B	Additional Related Work	27
C	Robertson–Webb Implementation Details	28
C.1	Queries on Ordered Pieces	28
C.2	Deterministic Choices	29
D	Omitted Proofs for Section 4	29
E	Full Pseudocode	31

1 Introduction

How should we divide a heterogeneous resource among n agents who value its parts differently? The cake cutting problem captures this question by modeling the resource as an interval and asking for a fair partition. Since Steinhaus formulated the problem in 1948, cake cutting has become a standard language for fair division in mathematics, economics, and computer science [28, 32]. In this paper, we focus on the fairness requirement of cake cutting: an allocation is *envy-free* if every agent values her own piece at least as much as every other agent’s piece.

Envy-free allocations have long been known to exist, and one short and elegant proof uses Sperner’s lemma [33, 35]. Finding one, however, is much harder. Since a protocol cannot read an arbitrary valuation in full, the standard *Robertson–Webb model* limits it to two kinds of queries—an evaluation query asking for the value of an interval, and a cut query for a point that cuts off a prescribed value.

We say a protocol is *finite* if it terminates for every collection of valuations, even when the number of queries depends on those valuations, and it is *bounded* if a function of n alone bounds that number.

1.1 Previous Work

For arbitrary nonatomic additive valuations, decades of work have produced only a few general results. For two agents, cut-and-choose solves the problem immediately. In the 1960s, Selfridge and Conway independently discovered a bounded envy-free protocol for three agents [12, 26].

For a larger number of agents, progress was much slower. In 1995, Brams and Taylor gave the first finite discrete envy-free protocol for any number of agents [11]. Their protocol always terminates, but its query count is not bounded by any function of n . Other finite protocols had the same limitation [25, 28]. On the lower-bound side, Procaccia proved in 2009 that every envy-free protocol requires $\Omega(n^2)$ queries [26]. The main open question was whether a bounded protocol existed at all.

Aziz and Mackenzie answered this question affirmatively in 2016 by designing a bounded protocol for four agents and then extending it to every n [3, 4]. Their protocol uses $n \uparrow\uparrow 6$ Robertson–Webb queries.¹

Throughout the paper, we call their construction the *AM protocol*. Our work builds on two of its main subroutines, *CORE* and *SUBCORE*, both of which we introduce in Section 3.

Before our work, the only known general improvement to AM’s upper bound was due to Sokolov, whose sharper analysis of the same protocol, especially of *CORE* and *SUBCORE*, reduces the query bound to $n^{8n^2(1+o(1))}$ [30].

We discuss additional related work in Appendix B.

1.2 Our Results and Techniques

In this paper, we reduce the general upper bound to a single exponential. More precisely, we prove the following.

Theorem 1.1 (Main Theorem). *For every $n \geq 1$ and every collection of nonnegative, additive, nonatomic valuations, there is a deterministic bounded Robertson–Webb protocol that returns a complete envy-free allocation using at most $n^{O(1)}2^n$ queries.*

¹Here $n \uparrow\uparrow k$ denotes a tower of k copies of n , evaluated from the top. For example, $n \uparrow\uparrow 2 = n^n$, and $n \uparrow\uparrow 3 = n^{n^n}$.

We summarize our improvement alongside the previously known general bounds in [Table 1](#) and briefly discuss our technical novelty below.

Result	Setting	Robertson–Webb Queries
Procaccia (2009) [26]	Lower Bound	$\Omega(n^2)$
Aziz–Mackenzie (2016) [3]	Upper Bound	$n \uparrow\uparrow 6$
Sokolov (2023) [30]	Upper Bound	$n^{8n^2(1+o(1))} = 2^{(8+o(1))n^2 \log_2 n}$
This paper	Upper Bound	$n^{O(1)}2^n$

Table 1: Comparison of the general Robertson–Webb query bounds.

Like Aziz and Mackenzie, our protocol runs recursively. At a high level, we give agents cake pieces without creating envy and call the unallocated cake the *residue*. We seek a nonempty group E that omits at least one agent, and we say that the omitted agents *dominate* E if no omitted agent would envy any member of E , even if that member (of E) received the entire residue, in addition to what she already had. *If this happens, then the omitted agents can keep their pieces and be gone, while we let agents in E recursively divide the residue.* Hence, our primary goal throughout recursion is to efficiently identify domination.

Aziz and Mackenzie’s protocol may produce $n \uparrow\uparrow 4$ partial allocations before establishing domination and recursing on fewer agents. Replacing this loop with polynomially many allocations, however, requires overcoming two challenges.

Challenge #1. First, suppose Alice values her own cake piece only slightly more than another piece P . Since her valuation is essentially unrestricted, this difference may be arbitrarily small. To establish domination, AM must make the residue worth no more to Alice than this difference, and reducing the residue to an arbitrarily small difference can be costly. AM achieves the reduction via repeatedly calling its CORE subroutine with Alice as the *cutter*. In each call, Alice divides the residue into pieces of an equal value to her. CORE allocates cake pieces to Alice and at least one other agent, and the unallocated pieces become the new residue. As Alice cuts the cake, CORE guarantees that her value for the residue is reduced by a fixed multiplicative factor. Still, reaching her arbitrarily small target may require arbitrarily many calls. To avoid these arbitrarily costly calls, AM uses an enormous fixed construction.

We improve AM’s procedure as follows. If Alice’s aforementioned difference is “tiny,” we add enough cake to P so that Alice is now willing to take it, instead of her original piece. We also ensure that this change is insignificant to every other agent, so everyone who already strongly prefers her own piece continues to do so. If, instead, the difference is “huge,” we use a polynomial number of CORE calls to make the residue small compared with it. We later show that only polynomially many such differences arise, and each requires this treatment at most once, so the entire process uses only polynomially many allocations and subroutine calls. (We also ensure that every such difference is either “tiny” or “huge.”) This resolves the first challenge.

Challenge #2. Resolving one of the differences in the previous challenge may require us to distribute a CORE allocation in a particular way, say, by giving its piece #42 to Alice. Resolving another difference may require us to distribute that same allocation differently, with the same piece going to Bob. Since one piece can go to only one agent, this allocation cannot resolve both differences. To address this, AM’s construction involves an enormous family of CORE allocations, so

that it carries out the two assignments in different allocations, and no actual cake piece is assigned twice.

We instead begin with only polynomially many CORE allocations and “prune” the ones that endanger us. Roughly speaking, we want the remaining allocations to be safe to use, in the sense that using one allocation to meet Alice’s need does not prevent us from finding another to meet Bob’s. Our key insight is that, if only a few allocations can resolve some difference, then they are too dangerous to rely on, because other assignments may use them up. Consequently, *we discard these allocations upfront* and later resolve this difference in another way. With careful bookkeeping, we show that after pruning, every difference still handled in this way can be resolved using many of the remaining allocations. Therefore, we can use a distinct allocation for each difference, thereby preventing all conflicts. With only polynomially many allocations, we achieve the same effect as AM’s enormous family: we meet Alice’s need in one allocation and Bob’s in another, without assigning any cake twice.

With these challenges resolved, by tracing the recurrence, we prove that our protocol gives the $2^{O(n)}$ Robertson–Webb query bound, as claimed.

2 Preliminaries

We model the cake as the interval $[0, 1]$ and let $\mathcal{A}$ be the finite set of agents. We assume each agent $i \in \mathcal{A}$ has a nonnegative, additive, nonatomic valuation V_i , which we call *i’s valuation*, and we allow a *piece* (of cake) to be a finite union of intervals. In other words, $V_i(A \cup B) = V_i(A) + V_i(B)$ for disjoint pieces A and B by additivity, and we may cut any interval to any intermediate value by nonatomicity. We call a piece that is to be divided on its own a *subcake*. For a set of agents $N \subseteq \mathcal{A}$ and a subcake C , an allocation $X = (X_i)_{i \in N}$ consists of pairwise disjoint pieces of C , and its *residue* is the unallocated part $R = C \setminus \bigcup_{i \in N} X_i$. The allocation is *complete* when the residue is empty (up to cut endpoints), and it is *envy-free* when $V_i(X_i) \geq V_i(X_j)$ for every $i, j \in N$.

In the *Robertson–Webb* model, a protocol accesses the valuations through two query types:

- $\text{Eval}_i(x, y)$ returns $V_i([x, y])$.
- $\text{Cut}_i(x, a)$, for $0 \leq a \leq V_i([x, 1])$, returns a point $y \geq x$ satisfying $V_i([x, y]) = a$.

We say a protocol is *bounded* if some function of the number of agents bounds its number of queries for every collection of valuations. Also, we note that the operations on disconnected pieces used later can be simulated by ordinary Robertson–Webb queries. We give this simulation and explain how we treat exact real-valued oracle answers in [Section C.1](#).

For a partial allocation X with residue R , we say that agent i *dominates* agent j if

$$V_i(X_i) - V_i(X_j) \geq V_i(R),$$

i.e., i would not envy j even if j receives all of R .

More generally, if a partial allocation X with residue R is envy-free and every agent in $N \setminus E$ dominates every agent in a nonempty set $E \subseteq N$, then any envy-free allocation of R within E can be added to X without making an agent outside E envy an agent in E . At a high level, this observation drives our protocol. We use it formally in [Lemma 4.1](#) and [corollary 4.2](#).

3 The Protocol at a High Level

Our protocol recursively reduces the number of *active* agents—ones who still need to divide the cake. A recursive call $\text{MAIN}(N, C)$ has the active agents N dividing the (sub)cake C . We design MAIN so that the result of each recursive call can be combined with other allocations without creating envy. The top-level call is $\text{MAIN}(\mathcal{A}, [0, 1])$, and calls with ≤ 3 active agents constitute base cases. In this section, we provide a high-level walkthrough of our protocol; we describe and analyze our protocol in full in [Section 4](#), and include the complete pseudocode in [Appendix E](#).

For a larger set of active agents N , we seek an envy-free partial allocation with residue R and a nonempty set $E \subseteq N$ such that every agent in $N \setminus E$ dominates every member of E . We let agents in $N \setminus E$ keep their pieces and recurse on (E, R) . In particular, recursion involves fewer active agents.

Throughout, we repeatedly use the AM subroutines CORE and SUBCORE [3].² The first subroutine, CORE , takes the current residue and m agents, with one agent designated as the *cutter*. The cutter first divides the residue into m pieces that she values equally. CORE then invokes SUBCORE , the second subroutine, to allocate cake from these pieces. In doing so, SUBCORE may give an agent one of these pieces in full or only part of one. Any cake cut off in this process becomes part of the new residue. At a high level, CORE produces an envy-free partial allocation in which the cutter and at least one other agent each receive one of these pieces in full. Since two of the cutter’s equal pieces are allocated in full, she values the new residue at most $(m - 2)/m$ as much as the previous one. In light of this observation, the main purpose of CORE calls is to “safely” shrink the value of the residue for a chosen agent. For instance, when we need to reduce Alice’s value for the residue by a certain factor, we use a *prescribed sequence* of repeated CORE calls on successive residues with the same set of agents and Alice as the cutter.

Later, we may encounter situations where we have given some cake to each agent in a particular group, but the agents in that group may still envy one another. When this happens, we need to redistribute this cake within the group so that the agents no longer envy one another, without creating envy between agents in different groups. Here, we use SUBCORE for a second purpose. For a group of r agents, we give SUBCORE the r pieces we need to redistribute, together with one empty piece, and specify each agent’s *required minimum*: the least value she must receive. Because these pieces may be disconnected, we fix an order on their interval components (for instance, from left to right; see [Section C.1](#)), and define a trim as removing a *prefix* and allocating the remaining *suffix*. SUBCORE finds an allocation without creating envy in which each agent receives a suffix of a different piece worth at least her required minimum. We give the formal guarantees in [Section 4.1](#).

3.1 One Recursive Call

Fix a call $\text{MAIN}(N, C)$, and write $n = |N|$ for its number of active agents.³ We construct the desired partial allocation in three stages.

Stage 1. Recall from Challenge #1 that the difference between the values an agent assigns to her own piece and another piece may be arbitrarily small. In Stage 1, we ensure that every such difference is either “tiny” or “huge,” with none in-between, so that we can handle it later as promised. We call this work *pre-processing*.

Pre-processing begins by going through the agents one at a time. For each agent, we make polynomially many consecutive CORE calls with her as the cutter. We store each envy-free allocation

²To be precise, we use a variant of SUBCORE that serves the same purpose but has a better query bound; see [Proposition 4.4](#).

³Throughout the analysis, n denotes the number of active agents in the current call. At the top level, $n = |\mathcal{A}|$.

returned by the CORE calls as a *snapshot*. In snapshot s , we call the cake allocated to agent k the **snapshot piece** $c_{s,k}$. For convenience later, we also call this same piece the **label- k piece** in snapshot s . We define agent i 's *bonus* over label k in snapshot s as her value for the label- i piece minus her value for the label- k piece, i.e., $V_i(c_{s,i}) - V_i(c_{s,k})$. Because each snapshot is envy-free, every bonus is nonnegative. Observe that Challenge #1 arises when a positive bonus is arbitrarily small.

We categorize bonuses as *small*, *large*, or *intermediate*. (The first two categories are what Challenge #1 informally called “tiny” and “huge.”) Our goal is to eliminate the intermediate bonuses. To this end, whenever agent i has an intermediate bonus, we run a prescribed sequence of CORE calls with i as the cutter. This shrinks the residue, so the bonus becomes large in comparison, and it remains large as the residue shrinks further. Since there are only polynomially many bonuses, we repeat this process at most polynomially many times. At the end, every bonus is either small or large.

Stage 2. Let R be the residue after Stage 1. If every agent values R at zero, we allocate it and finish. Otherwise, we choose a cutter who values R positively and study her snapshots.

The rough idea of Stage 2 is to find, for a pair of agents such as Alice and Bob, a snapshot piece that Bob values almost as much as his own, while Alice values it substantially less than hers. This way, when we later add a small amount of cake to that piece, Bob is willing to receive it, while Alice would not care about the addition, as she still values the resulting piece substantially less than her own.

Formally, for each snapshot s , we say agent i *marks* label k if she values the label- k piece $c_{s,k}$ substantially less than her own $c_{s,i}$. A *reassignment* gives each agent a distinct snapshot piece. We call snapshot s a *witness* for an ordered pair (i, j) of distinct agents if i marks some label k that j does not mark. Thus, a witness identifies a label we would like j to receive in a reassignment.

However, reassignments serving different pairs may conflict when they use the same snapshot, because this would require the same snapshot to be distributed in two different ways, which is clearly impossible. We therefore need to reserve distinct witness snapshots for different pairs to ensure that no such conflict arises.

To this end, we prune the “bad” snapshots. If a pair (i, j) has too few witnesses, we discard all of its witness snapshots, leaving that pair with no witness, and hence nothing to reserve. We repeat until every pair that still has a witness can reserve a distinct snapshot. If (i, j) is left with no witness, then in every remaining snapshot, j marks every label that i marks.

For each snapshot that remains after pruning, we form a graph G_s with an edge $a \rightarrow k$ when a does **not** mark k . These edges describe the reassignments we can use in Stage 3. Every self-loop is therefore present.

We now check for a potential shortcut. If some G_s is not strongly connected, we choose a sink strongly connected component S . No edge leaves S , so every agent in S marks every label outside S . We can then run prescribed sequences of CORE calls for the agents in S , shrinking the residue until S dominates $N \setminus S$. We simply recurse on $N \setminus S$ and return.

Otherwise, every G_s is strongly connected. We say an agent's marks across the remaining snapshots form her *profile*, and agents with equal profiles form a *profile class*. We later show that at least two profile classes remain, and enter Stage 3 with enough witness snapshots to avoid conflicts.

Stage 3. In this final stage, our goal is to build an envy-free partial allocation X from which we can reach domination. For agents i, j in different profile classes, we want to make $V_i(X_i) - V_i(X_j)$ positive whenever i values R positively, so that once we shrink $V_i(R)$ to no more than this difference, i dominates j . We first establish these comparisons, distinguishing whether (i, j) has a witness, and then handle agents within the same profile class.

Recall every G_s is now strongly connected. First suppose (i, j) is witnessed by snapshot s through a label k marked by i but not j . The edge $j \rightarrow k$ lies on a directed cycle whose edges, together with self-loops outside the cycle, form a perfect matching. The matching pairs every agent with a distinct label she does *not* mark; in particular, it pairs j with label k . Moreover, we reserve a different witness snapshot for every pair that has a witness, so the reassignments do not conflict. See Figure 1.

For every witness snapshot s we reserve and every label k , we cut a small piece $A_{s,k}$ from the residue and attach it to $c_{s,k}$. We choose the added cake so that an agent who does not mark k is willing to receive the resulting **enlarged snapshot piece** $c_{s,k} \cup A_{s,k}$, while an agent who marks k still prefers her own piece. In particular, j is willing to receive this piece, while i still prefers her own.

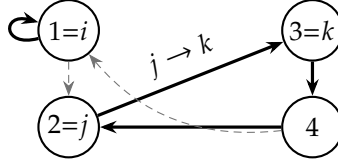


Figure 1: A matching built from a witness. Here $i = 1$ marks label $k = 3$, while $j = 2$ does not. The solid arrows show the matching: agent 1 receives the label-1 piece, agent 2 receives the label-3 piece, agent 3 receives the label-4 piece, and agent 4 receives the label-2 piece. In particular, j receives the label- k piece, as desired.

Suppose instead that (i, j) has no witness, so every label marked by i is also marked by j in every remaining snapshot. In this case, we instead use the residue to make i value her own share more than j 's. Specifically, we cut one small piece P_h for every agent h , with sizes chosen so that P_i is worth sufficiently more than P_j to i .

It remains to address agents within the same profile class, as they may still envy one another. We handle each profile class separately. Within one class, we use SUBCORE to redistribute the cake currently assigned to its agents. Recall from the beginning of Section 3 that this step requires us to specify each agent's *required minimum*, the least value she must receive. Here, we choose the required minima so that every agent still receives enough value to preserve her comparisons with agents in other classes. We also place the snapshot pieces last in each share we give to SUBCORE. Doing so ensures that every trim occurs before the snapshot pieces and leaves them intact. SUBCORE therefore removes envy within each class without undoing the comparisons between classes.

We finally have the long-wanted envy-free partial allocation that we need for a recursion. Choose a profile class $E \subseteq N$. For each agent $a \in N \setminus E$ who values the remaining residue positively, we use a prescribed sequence of CORE calls to shrink her value for the remaining residue to at most $V_a(X_a) - V_a(X_j)$, for every agent $j \in E$. Agents in $N \setminus E$ who value the remaining residue at zero already dominate every member of E . Furthermore, only E remains active. Therefore, we return the current partial allocation and recurse on E with the final residue.

4 Proof of the Main Theorem

4.1 Basic Facts

We begin by defining a handful of shorthand notations and stating several facts that we frequently use throughout the main proof.

For allocations X and Y of disjoint subcakes, we define $X \oplus Y$ by

$$(X \oplus Y)_i = X_i \cup Y_i \quad \text{for every agent } i.$$

If either allocation omits an agent, we treat that agent as receiving the empty piece. We use $\oplus$ to combine several allocations the same way. For a partial allocation X and agents i, j , define the *difference in value* by

$$\Delta_{ij}(X) = V_i(X_i) - V_i(X_j).$$

We use the following facts whenever the protocol adds partial allocations or completes a recursive branch.

Lemma 4.1. *The following hold.*

- (i) *Suppose several partial allocations use pairwise disjoint subcakes and are envy-free among the same participating agents. Giving every agent the union of her pieces preserves envy-freeness. Moreover, as these allocations are added, the difference in value for any pair of participating agents cannot decrease.*
- (ii) *Once agent i dominates agent j , every later allocation of part of the residue preserves this domination.*

Proof. For (i), each constituent allocation has a nonnegative difference in value for every pair of participating agents, so summing over the disjoint subcakes proves both claims. For (ii), let R be the old residue, and suppose the later allocation gives pieces Y_i and Y_j to i and j and leaves residue R' . Using $\Delta_{ij}(X) \geq V_i(R)$ and additivity, we have

$$V_i(X_i \cup Y_i) - V_i(X_j \cup Y_j) \geq V_i(R) + V_i(Y_i) - V_i(Y_j) \geq V_i(R').$$

The final inequality holds because R' is disjoint from Y_j and contained in $R \setminus Y_j$. $\square$

Corollary 4.2. *Suppose a partial allocation X is envy-free and every agent in $N \setminus E$ dominates every agent in a nonempty set $E \subseteq N$. Any complete envy-free allocation of the residue within E can be combined with X to obtain a complete envy-free allocation among all agents in N .*

Proof. By Lemma 4.1 (ii), agents outside E do not envy agents in E , even after the latter receive the residue. Agents in E cannot begin to envy an outside agent because only members of E receive more cake. Finally, by Lemma 4.1 (i), envy-freeness within E is preserved, while comparisons within $N \setminus E$ remain unchanged. $\square$

We use the following formal guarantees of the CORE subroutine of Aziz and Mackenzie and our variant of their SUBCORE subroutine [3].

CORE. A call $\text{CORE}(N, R, c)$ takes a residue R , a set N of n agents, and a designated *cutter* $c \in N$. It returns an envy-free partial allocation X and a new residue R' consisting of the unallocated part of R . The cutter and at least one other agent receive whole pieces valued at $V_c(R)/n$ by the cutter.

We use CORE mainly for two purposes: to collect a fixed number of partial allocations or, more importantly, to reduce the value of the residue with respect to the cutter. A *prescribed sequence* repeats CORE with the same agents and cutter on successive residues, with a maximum number of calls fixed in advance. When the purpose is to collect allocations, the sequence continues until it has made the prescribed number of calls. When the purpose is only to reduce the cutter's residue value, it stops early if that value reaches zero.

SUBCORE. A SUBCORE call used inside CORE takes m agents and at least $m + 1$ pieces, and returns an envy-free allocation in which the agents receive possibly trimmed parts of distinct pieces.

We later call SUBCORE directly in a more specific setting. There, the call receives m agents, m pieces $W_1, \dots, W_m$, one empty piece (making it $m + 1$ total), and a *required minimum* $q_i \geq 0$ for every agent i . The required minimum is the least value that the agent may receive. We store the interval components of each W_h in a fixed order, so every trim removes a prefix and leaves a suffix as defined in [Section C.1](#). Finally, every unused prefix returns to the residue.

Proposition 4.3. *The following properties hold.*

(i) *A call $\text{CORE}(N, R, c)$ returns an envy-free partial allocation X and a residue R' , and the cutter and at least one other agent receive whole pieces valued at $V_c(R)/n$ by the cutter.*

(ii) *The returned residue satisfies*

$$V_c(R') \leq \frac{n-2}{n} V_c(R).$$

(iii) *For $n \geq 3$ and $\beta \geq 1$, a prescribed sequence used to reduce the cutter's residue value, with maximum length $\lceil \log_{n/(n-2)} \beta \rceil$, leaves a residue R' satisfying*

$$V_c(R') \leq \frac{V_c(R)}{\beta},$$

where R is the residue at the start of the sequence.

Proof. Part (i) follows from [3]'s proof of CORE; we prove that the same guarantee holds for our SUBCORE variant in [Appendix D](#). Assuming (i), (ii) follows immediately, because two of the pieces worth $V_c(R)/n$ are allocated. Applying this repeatedly yields (iii), as

$$V_c(R') \leq \left(\frac{n-2}{n} \right)^{\lceil \log_{n/(n-2)} \beta \rceil} V_c(R) \leq \frac{V_c(R)}{\beta}.$$

(If the sequence stops early, then $V_c(R') = 0$, so the same bound is immediate.) □

Proposition 4.4. *Our protocol uses a variant of AM's SUBCORE that serves the same purpose as in AM's protocol. Every call to this variant with m agents uses $O(2^m)$ cut queries and $O(m2^m)$ evaluation queries to those agents.⁴ A CORE call on n agents uses $O(2^n)$ cut queries and $O(n2^n)$ evaluation queries. We defer description of our SUBCORE as well as the proof to [Appendix D](#).*

We now state a guarantee of SUBCORE with conditions tailored to our needs, and we also defer its proof to [Appendix D](#).

Proposition 4.5. *Let E be a set of m agents. Consider a SUBCORE call with agents E , the m ordered pieces $(W_h)_{h \in E}$, the empty piece $\emptyset$, and required minima $q_i \geq 0$ for $i \in E$ satisfying*

$$V_i(W_h) \geq q_i \quad \text{for } i, h \in E.$$

The call uses the tie-breaking convention described in [Section C.2](#), under which any tie between a piece W_h and the empty piece is broken in favor of W_h . Then, SUBCORE returns an envy-free allocation in which every agent receives a suffix of a distinct W_h worth at least her required minimum, while the empty piece remains unallocated.

⁴For the original SUBCORE, Sokolov's analysis gives a query bound with base $(3 + \sqrt{5})/2 \approx 2.618$ [30].

4.2 A Lemma for Cutting Several Pieces at Once

Recall that in Stage 3 of [Section 3.1](#), we needed to make the differences in value positive so that our algorithm could recurse, and we described two ways to achieve this. When a pair (i, j) has a witness, we add a small amount of cake to the snapshot piece assigned to j . When it has no witness, we instead use pieces from the residue to make i value her own share more than j 's. To carry out both steps, we need to cut many disjoint pieces from the same residue and ensure that each has roughly the right value for every agent. The following lemma shows that we can do this efficiently.

Lemma 4.6. *Let N be a set of n agents, let R be an ordered piece whose component values are stored for every agent, and let J be a nonempty finite index set. For each $j \in J$, let $\lambda_j \geq 0$, and suppose*

$$\sum_{j \in J} \lambda_j \leq 1.$$

Let $0 < \delta < 1$. There is a deterministic bounded procedure that produces pairwise disjoint pieces $C_j \subseteq R$, one for each $j \in J$, such that, for every $i \in N$ and $j \in J$,

$$|V_i(C_j) - \lambda_j V_i(R)| \leq \delta V_i(R).$$

Furthermore, this procedure uses $O(n^2 \delta^{-2} \log(n|J| + 1))$ Robertson–Webb queries.

Proof. Let $m = |J|$ and set $\gamma = \delta^2 / (2 \log(2nm + 2))$.

We ask each agent i with $V_i(R) > 0$ to cut R , following its fixed order (recall the convention from [Section C.1](#)), into pieces worth $\gamma V_i(R)$ to her, except possibly the last piece, which may be smaller. We combine the points where the agents cut R with the endpoints of the intervals that make up R . Together, these points divide R into smaller pieces, which we call *cells*. Whenever an agent makes one of these cuts, we use [Proposition C.1](#) to record every agent's value for the two resulting intervals. Consequently, once all cuts are made, we know every agent's value for every cell. By construction, every cell K satisfies $V_i(K) \leq \gamma V_i(R)$ for every agent i .

We next show that we can form the desired pieces from these cells. Independently for each cell, we make one choice: place it in C_j with probability λ_j for each $j \in J$, or leave it unused with probability $1 - \sum_{j \in J} \lambda_j$. Fix an agent i with $V_i(R) > 0$ and some $j \in J$, and let $x_K = V_i(K) / V_i(R)$, i.e., the fraction of i 's value for R contained in cell K . Because the cells partition R , we have $\sum_K x_K = 1$. Moreover, $x_K \leq \gamma$ for every cell, and hence

$$\sum_K x_K^2 \leq \gamma \sum_K x_K = \gamma.$$

The fraction of i 's value placed in C_j is the sum of x_K over the cells placed in C_j , and its expected value is λ_j . Therefore, Hoeffding's inequality gives

$$\Pr \left[\left| \sum_K x_K \mathbf{1}_{\{K \text{ is placed in } C_j\}} - \lambda_j \right| > \delta \right] \leq 2 \exp \left(- \frac{2\delta^2}{\sum_K x_K^2} \right) \leq 2e^{-2\delta^2/\gamma}.$$

There are at most nm choices of i and j . It follows that the probability that any of the desired inequalities fails is at most

$$2nm e^{-2\delta^2/\gamma} = \frac{2nm}{(2nm + 2)^4} < 1.$$

Thus, we can place the cells so that $|V_i(C_j) - \lambda_j V_i(R)| \leq \delta V_i(R)$ for every $i \in N$ and $j \in J$. The pieces C_j are pairwise disjoint because every cell is placed in at most one of them.

It remains to find one way to place the cells. Since there are finitely many cells and we know every agent's value for each one, we can check all possibilities until we find one for which every inequality holds. The argument above shows that one exists, and this search uses no further queries.⁵

Finally, we count the queries used to create the cells and record their values. Each agent needs $O(1/\gamma)$ Robertson–Webb cut queries to divide R into her small pieces, so all agents together need $O(n/\gamma)$ such queries. By [Proposition C.1](#), recording every agent's value after each cut costs $O(n)$ evaluation queries. Thus, the procedure uses $O(n^2/\gamma) = O(n^2 \delta^{-2} \log(n|J| + 1))$ queries, as claimed. $\square$

4.3 Pre-processing the Snapshots

We now begin our analysis of the full algorithm. The first step, as outlined in Stage 1 of [Section 3.1](#), is pre-processing, which involves eliminating the “intermediate” bonuses so that we can later resolve Challenge #1. In this section, we describe pre-processing in detail.

Fix a call $\text{MAIN}(N, C)$, where C is formed from finitely many intervals, and (with slight abuse of notation) set $n = |N|$.

For $n = 1$, we give the subcake to the sole agent. For $n = 2$, we use cut-and-choose: one agent divides the subcake into two pieces she values equally, the other chooses a piece she prefers, and the first agent receives the remaining piece. For $n = 3$, we use the Selfridge–Conway protocol, which requires at most 14 Robertson–Webb queries [\[1, 12\]](#). When C has several interval components, we apply these protocols to the fixed order defined in [Section C.1](#) and implement the resulting queries as explained there [\[28\]](#). We henceforth assume $n \geq 4$ and set $Q = n(n - 1)$.

Our first step is to construct the snapshots. Throughout, we use U for the residue we currently have, H for the partial allocation we have constructed so far, and $\mathcal{S}$ for the collection of snapshots. Initially, $U = C$, H is the empty allocation, and $\mathcal{S} = \emptyset$. To begin pre-processing, we go through the agents one at a time and, for each agent, make Q^2 consecutive CORE calls with her as the cutter. Each call uses the residue left by the one before it. Altogether, these calls return nQ^2 envy-free allocations. We store them in $\mathcal{S}$, index them by s , and call each one a *snapshot*. For convenience, we name each piece in a snapshot in two ways. In snapshot s , we call the cake allocated to agent k both her *snapshot piece* $c_{s,k}$ and the *label- k piece*. Note that this label stays with the snapshot piece if it is later reassigned.

In snapshot s , for agent i , define

$$b_{s,i} = V_i(c_{s,i}), \quad d_{s,i,k} = b_{s,i} - V_i(c_{s,k}).$$

We call $d_{s,i,k}$ agent i 's *bonus* over label k . Because snapshot s is envy-free, this bonus is nonnegative. For any collection $\mathcal{T}$ of snapshots, we write $\text{Id}(\mathcal{T})$ for the identity assignment, in which agent i receives the label- i piece from every snapshot in $\mathcal{T}$; that is, $\text{Id}(\mathcal{T})_i = \bigcup_{s \in \mathcal{T}} c_{s,i}$.

For each agent i , set

$$\Gamma_i(U) = \frac{V_i(U)}{Q^2}, \quad \eta_i(U) = \frac{\Gamma_i(U)}{4n(Q + 1)}, \quad \epsilon_i(U) = \frac{\eta_i(U)}{n},$$

and recompute these quantities whenever U shrinks. We call a bonus *small* when it is at most $\epsilon_i(U)$, *intermediate* when it lies strictly between $\epsilon_i(U)$ and $\Gamma_i(U)$, and *large* when it is at least $\Gamma_i(U)$.

⁵We discuss ways to avoid this exhaustive search in [Section 5](#), though these alternatives concern the computation between queries and do not affect the Robertson–Webb query bound.

Whenever an intermediate bonus exists and belongs to agent i , we run a prescribed sequence of at most $\left\lceil \log_{n/(n-2)}(4n^2(Q+1)) \right\rceil$ CORE calls with i as the cutter. We add the returned allocations to H , replace U with the residue left by the sequence, and recompute the quantities above. We repeat this process until no intermediate bonus remains, so that every bonus is either small or large, and call the resulting residue R^* .

Lemma 4.7. *The construction returns a partial allocation H , a collection $\mathcal{S}$ of nQ^2 snapshots, and a residue R^* such that:*

- (i) H and every snapshot are envy-free, and their allocated pieces together with R^* partition C ;
- (ii) every bonus satisfies

$$d_{s,i,k} \leq \epsilon_i(R^*) \quad \text{or} \quad d_{s,i,k} \geq \Gamma_i(R^*).$$

Furthermore, pre-processing uses $n^{O(1)}2^n$ Robertson–Webb queries.

Proof. We first prove (i). By Proposition 4.3 (i), every CORE call returns an envy-free allocation. Because each call uses the residue left by the preceding one, all these allocations use pairwise disjoint cake. We store the first nQ^2 allocations as snapshots and add every later allocation to H . Thus, every snapshot is envy-free, and H is envy-free by Lemma 4.1 (i). Moreover, after each call, the cake allocated by that call together with the new residue is the residue we had before the call. It follows that the pieces allocated in H and the snapshots, together with the final residue R^* , partition C . This proves (i).

We next show that pre-processing terminates with no intermediate bonus. Suppose $d = d_{s,i,k}$ is the intermediate bonus being processed while the current residue is U . By definition, $\epsilon_i(U) < d < \Gamma_i(U)$. By Proposition 4.3 (iii), the sequence with factor $4n^2(Q+1)$ leaves a residue U' satisfying $V_i(U') \leq \frac{V_i(U)}{4n^2(Q+1)}$. Thus,

$$\Gamma_i(U') = \frac{V_i(U')}{Q^2} \leq \frac{V_i(U)}{4n^2(Q+1)Q^2} = \epsilon_i(U) < d.$$

Therefore, d is large after this step. The snapshot pieces do not change during pre-processing, so neither does d . Every later residue is contained in U' , so the value a bonus must reach to be large can only decrease. Thus, d remains large for the rest of pre-processing.

Each of the nQ^2 snapshots has Q bonuses $d_{s,i,k}$ with $k \neq i$. Thus, at most nQ^3 bonuses can become intermediate. Each step makes one of them permanently large, so the same bonus is never selected again. The process therefore stops after at most nQ^3 steps, with no intermediate bonus remaining. This proves (ii).

Finally, we count the queries. Constructing the snapshots takes nQ^2 CORE calls, and evaluating every snapshot piece for every agent takes $O(n^3Q^2)$ queries. By the argument above, there are at most nQ^3 later sequences, each containing at most

$$\left\lceil \log_{n/(n-2)}(4n^2(Q+1)) \right\rceil = O(n \log n)$$

CORE calls. Since $Q = n(n-1)$, we make only polynomially many CORE calls. Combining this with Propositions C.1 and 4.4 gives the claimed query bound. $\square$

If every agent values R^* at zero, we add R^* to one agent's share in $H \oplus \text{Id}(\mathcal{S})$ and return that allocation. By Lemmas 4.1 and 4.7, this allocation is complete and envy-free.

4.4 Pruning the Snapshots

With pre-processing complete, we now begin Stage 2 of [Section 3.1](#), where we prune the snapshots so that the reassignments chosen later do not conflict. For the rest of the proof, assume that some agent values R^* positively. For every agent i , we abbreviate

$$\Gamma_i = \Gamma_i(R^*), \quad \eta_i = \eta_i(R^*), \quad \epsilon_i = \epsilon_i(R^*).$$

We now give “mark” its formal meaning. In snapshot s , let $F_{s,i}$ be the set of labels that agent i marks, i.e.,

$$F_{s,i} = \{k \in N : d_{s,i,k} > \epsilon_i\}.$$

By [Lemma 4.7](#), if agent i marks label k in snapshot s , then her bonus $d_{s,i,k}$ over that label is large. Conversely, if she does not mark label k , then this bonus is small.

We choose an agent c with $V_c(R^*) > 0$ and begin with the Q^2 snapshots made with c as the cutter. Recall that snapshot s witnesses the pair (i, j) if i marks some label that j does not. In our notation, this means $F_{s,i} \not\subseteq F_{s,j}$. Such a witness identifies a label that we may later want to give to j . This also brings us back to Challenge #2 from the introduction. The same snapshot may witness several pairs, and those pairs may require us to distribute that snapshot in different, conflicting ways. To resolve this issue, we need to reserve a different witness snapshot for each pair.

We achieve this by pruning the snapshots. Observe that, since there are Q (ordered) pairs of agents, if every pair has at least Q witnesses, then our goal is trivially met by the pigeonhole principle. Indeed, we can go through the pairs one at a time. Before we reach a pair, at most $Q - 1$ snapshots have been reserved, so at least one witness remains unused. In light of this observation, whenever a pair has between 1 and $Q - 1$ witnesses, we discard all of its witness snapshots. We repeat until no such pair remains and call the remaining collection $\mathcal{P}$. Thus, in $\mathcal{P}$, every pair either has no witness or has many, enough to avoid conflicts later.

We write agent i 's profile as $\mathbf{F}_i = (F_{s,i})_{s \in \mathcal{P}}$. Agents with the same profile form a *profile class*. We write $\mathbf{F}_i \leq \mathbf{F}_j$ when $F_{s,i} \subseteq F_{s,j}$ for every $s \in \mathcal{P}$, i.e., no snapshot in $\mathcal{P}$ witnesses (i, j) . We write $\mathbf{F}_i < \mathbf{F}_j$ when, in addition, $F_{s,i} \subsetneq F_{s,j}$ for some $s \in \mathcal{P}$.

Later, we recurse on one profile class. We need at least two profile classes to remain, so that the recursive call contains fewer agents. The following fact about the cutter prevents all agents from having the same profile.

Lemma 4.8. *If agent c satisfies $V_c(R^*) > 0$, then $F_{s,c} \neq \emptyset$ in every snapshot s made with c as the cutter.*

Proof. Fix one such snapshot, and let U and T be the residues immediately before and after its CORE call. Agent c divides U into n pieces of equal value, each worth $b_{s,c}$ to her. The allocated snapshot pieces together with T partition U . Therefore,

$$V_c(T) = \sum_{k \in N} [b_{s,c} - V_c(c_{s,k})] = \sum_{k \in N} d_{s,c,k}.$$

Because snapshot s is envy-free, every term in this sum is nonnegative. Moreover, by [Proposition 4.3](#) (i), c and another agent each receive a whole piece cut by c . Both pieces are worth $b_{s,c}$ to c , so the corresponding two terms are zero. Thus, at most $n - 2$ terms are positive.

All later operations only shrink the residue, so $R^* \subseteq T$. It follows that some label k satisfies

$$d_{s,c,k} \geq \frac{V_c(T)}{n-2} \geq \frac{V_c(R^*)}{n-2} > \frac{V_c(R^*)}{Q^2} = \Gamma_c,$$

where we used $V_c(R^*) > 0$ and $Q^2 > n - 2$. Since $\Gamma_c > \epsilon_c$, agent c marks label k . Hence $F_{s,c} \neq \emptyset$. $\square$

Lemma 4.9. *The collection $\mathcal{P}$ contains at least Q snapshots. For every pair (i, j) , the condition $F_{s,i} \subseteq F_{s,j}$ either holds throughout $\mathcal{P}$ or fails in at least Q snapshots. Moreover, at least two profile classes remain.*

Proof. When a pair (i, j) has between 1 and $Q - 1$ witnesses, the procedure deletes its entire witness set. After that deletion, this pair has no witnesses and cannot regain one as the collection shrinks, so each of the Q pairs causes at most one deletion. Every deletion removes at most $Q - 1$ snapshots. Consequently, the final collection satisfies $|\mathcal{P}| \geq Q^2 - Q(Q - 1) = Q$, and every surviving witness set is either empty or has size at least Q .

Suppose all agents had the same profile. Then, in every snapshot $s \in \mathcal{P}$, they would all mark the same labels. Agent i never marks label i , so no other agent could mark label i either. This holds for every i , so no agent could mark any label in s . However, s was made with c as the cutter, and Lemma 4.8 says that c marks at least one label in s , a contradiction. Thus, at least two profile classes remain. $\square$

To sum up, after pruning, agents in different profile classes fall into one of two cases. Either $F_i \not\subseteq F_j$, in which case at least Q snapshots witness (i, j) , or $F_i < F_j$, in which case no snapshot in $\mathcal{P}$ witnesses (i, j) .

4.5 Constructing the Partial Allocation

Having pruned the snapshots, we now enter Stage 3, where the main goal is to construct the partial allocation needed for recursion. This section is rather lengthy, so we give a brief outline in Figure 2. First, we address agents in different profile classes. Our goal, roughly speaking, is to distribute the snapshot pieces and give agents additional cake so that each agent values her own share more than the share of anyone in another profile class.⁶ There are two subcases, and we develop a tool (Lemmas 4.11 and 4.13) for each one. We then use both tools in one construction that handles the two (sub)cases together. Finally, we take care of agents within the same class, where, to ensure no envy exists, we use SUBCORE to redistribute the cake these agents receive.

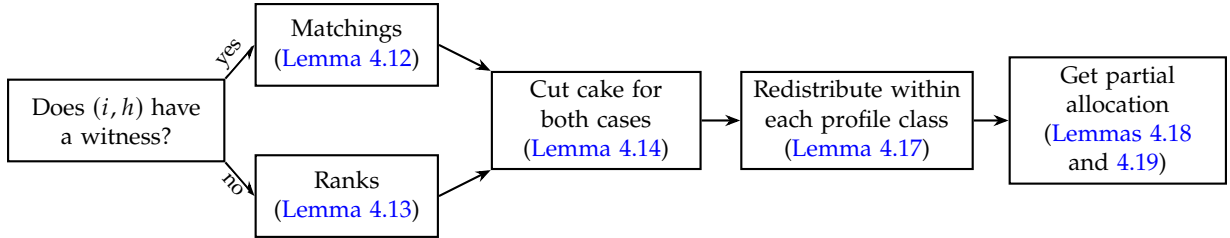


Figure 2: A flowchart for Stage 3.

To begin, for every snapshot s that survives the pruning, we define a directed graph G_s . The graph contains the edge $i \rightarrow k$ if $k \notin F_{s,i}$, and in particular every self-loop is present. We first look for a potential shortcut.

Lemma 4.10. *Suppose some G_s is not strongly connected, and let S be a sink strongly connected component of G_s . Then we can construct an envy-free partial allocation in which every agent in S dominates every agent in $N \setminus S$. We can therefore recurse on $N \setminus S$.*

⁶Formally, every agent who values R^* positively strictly prefers her own share to the share of every agent in another profile class. See Lemma 4.19.

Proof. Since G_s is not strongly connected, $S \subsetneq N$, and no edge of G_s leaves S . Thus, for every $i \in S$ and $j \in N \setminus S$, the label j belongs to $F_{s,i}$. By Lemma 4.7, we have

$$V_i(c_{s,i}) - V_i(c_{s,j}) = d_{s,i,j} \geq \Gamma_i = \frac{V_i(R^*)}{Q^2}.$$

Set $Y_0 = H \oplus \text{Id}(S)$. Since H and every snapshot in $\mathcal{S}$ are envy-free, Lemma 4.1 (i) gives $\Delta_{ij}(Y_0) \geq \Gamma_i$. For each $i \in S$ with $V_i(R^*) > 0$, we repeatedly call CORE with i as the cutter, each time using the residue left by the previous call, until the residue is worth at most $V_i(R^*)/Q^2 = \Gamma_i$ to her. Let Y be the allocation obtained by adding every allocation returned by these calls to Y_0 . Once the sequence for i ends, she values the residue at most Γ_i , and later calls only shrink it. Since every returned allocation is envy-free, Lemma 4.1 (i) gives $\Delta_{ij}(Y) \geq \Delta_{ij}(Y_0) \geq \Gamma_i$ for every $j \in N \setminus S$.

If $i \in S$ satisfies $V_i(R^*) = 0$, the remaining residue is worth zero to her. Since Y is envy-free, she also dominates every $j \in N \setminus S$.

Every agent in S therefore dominates every agent in $N \setminus S$. By Corollary 4.2, we recurse on $N \setminus S$. $\square$

For the rest of the proof, we assume that every G_s is strongly connected. Let (i, h) be a pair of agents from different profile classes. As outlined earlier, our goal is to ensure that i values her own share more than h 's, and we prove this in two cases.

Different profile classes, Case 1. Suppose $\mathbf{F}_i \not\leq \mathbf{F}_h$. Then some snapshot s witnesses (i, h) ; choose a label $k \in F_{s,i} \setminus F_{s,h}$.

We want to give the label- k piece to h , since i values this piece substantially less than her own piece, while h values it almost as much as her own. Since h does not mark k , the graph G_s contains the edge $h \rightarrow k$. The following lemma shows that we can extend this edge to a perfect matching using only edges of G_s .

Lemma 4.11. *Every edge $j \rightarrow k$ of a strongly connected graph G_s belongs to a perfect matching that pairs each agent with a distinct label and uses only edges of G_s .*

Proof. If $j = k$, the identity matching works. Otherwise, strong connectivity gives a simple directed path from k to j . This path together with $j \rightarrow k$ forms a directed cycle. The cycle edges and the self-loops at all vertices outside the cycle form the required perfect matching. $\square$

We now need to choose such a matching for every pair (i, h) with $\mathbf{F}_i \not\leq \mathbf{F}_h$, while ensuring that different pairs use different witness snapshots. Let $\mathcal{J}$ denote the collection of snapshots we select, initially empty. We go through the pairs one at a time. For each pair (i, h) , we choose an unused witness snapshot s and a label $k \in F_{s,i} \setminus F_{s,h}$. By Lemma 4.11, we can choose a perfect matching μ_s containing the edge $h \rightarrow k$. We include s in $\mathcal{J}$ and use this matching for s , under which h receives the label- k piece.

If $\mathcal{J}$ contains fewer than Q snapshots after we have considered every pair, we choose additional unused snapshots and give each its identity matching until $\mathcal{J}$ contains exactly Q snapshots.

Lemma 4.12. *Assume every G_s with $s \in \mathcal{P}$ is strongly connected. We can select a collection $\mathcal{J}$ of Q distinct snapshots and a perfect matching μ_s for each $s \in \mathcal{J}$. Each matching uses only edges of G_s , so every agent is paired with a label she does not mark. Moreover, for every pair (i, h) with $\mathbf{F}_i \not\leq \mathbf{F}_h$, some $s \in \mathcal{J}$ satisfies $\mu_s(h) \in F_{s,i} \setminus F_{s,h}$.*

Proof. There are at most Q such pairs. By Lemma 4.9, each such pair has at least Q witness snapshots in $\mathcal{P}$. When we reach a pair (i, h) , fewer than Q snapshots have been selected, so an unused witness remains. In that snapshot, Lemma 4.11 gives a perfect matching that gives h a label marked by i but not by h . Thus, we can handle every pair using a different snapshot.

If fewer than Q snapshots are selected, then $|\mathcal{P}| \geq Q$ by Lemma 4.9, so enough unused snapshots remain to bring the total to Q . We give each added snapshot its identity matching. Every self-loop belongs to G_s , so these matchings also use only edges of G_s . $\square$

Different profile classes, Case 2. Suppose now that $\mathbf{F}_i < \mathbf{F}_h$. Let $\mathcal{E}$ be the set of profile classes. Overloading the notation, we write $\mathbf{F}_E$ for the profile shared by agents in E . Our key tool for this case is the notion of *rank*. For each class E , define its *rank* by

$$\ell_E = \max \left\{ t : \begin{array}{l} \exists E_0, E_1, \dots, E_t \in \mathcal{E} \text{ with } E_0 = E, \\ \mathbf{F}_{E_0} < \mathbf{F}_{E_1} < \dots < \mathbf{F}_{E_t} \end{array} \right\},$$

and set $\ell_h = \ell_E$ for every $h \in E$. The following lemma explains the name: if $\mathbf{F}_E < \mathbf{F}_D$, then E has higher rank than D .

Lemma 4.13. *For every profile class E and every agent $h \in E$,*

$$0 \leq \ell_h \leq n - 1, \quad \sum_{h \in N} \ell_h \leq Q.$$

Moreover, for profile classes E, D , if $\mathbf{F}_E < \mathbf{F}_D$, then $\ell_E \geq \ell_D + 1$.

Proof. Every chain in the definition of ℓ_E consists of distinct profile classes and therefore contains at most n classes. Hence, $0 \leq \ell_h \leq n - 1$ for every h , and summing over the n agents gives $\sum_{h \in N} \ell_h \leq n(n - 1) = Q$. If $\mathbf{F}_E < \mathbf{F}_D$, prepending E to a longest chain starting at D produces a chain of length $\ell_D + 1$ starting at E , so $\ell_E \geq \ell_D + 1$. $\square$

Bringing the two cases together. With both tools in place, we now build one partial allocation that handles both cases. For Case 1, we slightly enlarge the snapshot pieces assigned by the matchings. We choose each enlargement so that an agent who does not mark the label is willing to receive the piece, while an agent who marks the label still values it substantially less than her own piece. For Case 2, we cut one piece from R^* for each agent, with its size determined by the agent's rank. We choose all these pieces to be disjoint and now define their sizes.

Set $L = 1/(nQ^2)$ and $\delta = L/(16n(Q + 1))$. For every $s \in \mathcal{J}$ and every label k , we also set

$$r_{s,k} = \max \left\{ 0, \frac{d_{s,i,k}}{V_i(R^*)} \text{ where } i \in N, V_i(R^*) > 0, d_{s,i,k} \leq \epsilon_i \right\},$$

i.e., the largest fraction of R^* needed to cover any small bonus over label k in snapshot s . Using these values, we ask for two kinds of pieces. For every such s and k , we ask for a piece $A_{s,k}$ of size $r_{s,k} + \delta$, and call it the *addition* to $c_{s,k}$. For every agent h with $\ell_h > 0$, we ask for a piece P_h of size $L(\ell_h - \frac{1}{2}) + \delta$, and call it the *rank piece* for h . When $\ell_h = 0$, we set $P_h = \emptyset$. We feed these requests into Lemma 4.6, using R^* and δ .

Lemma 4.14. *The pieces $A_{s,k}$ and P_h are pairwise disjoint subsets of R^* with the following properties:*

- (i) *if $d_{s,i,k} \leq \epsilon_i$, then $V_i(A_{s,k}) \geq d_{s,i,k}$*

(ii) for every $i \in N$, $s \in \mathcal{J}$, and $k \in N$, $V_i(A_{s,k}) \leq \eta_i$;

(iii) for every $i, h \in N$, $\max\{0, L(\ell_h - 1/2)\}V_i(R^*) \leq V_i(P_h) \leq L\ell_h V_i(R^*)$.

They can be produced using polynomially many Robertson–Webb queries. We defer the proof to [Appendix D](#).

(iii) already gives us the bounds on the rank pieces needed for Case 2. We now verify what the additions do, first within each snapshot. If agent i does not mark label k , attaching $A_{s,k}$ to $c_{s,k}$ makes the enlarged piece worth at least as much as her own snapshot piece. If she marks label k , the enlarged piece remains worth substantially less than her own snapshot piece.

Lemma 4.15. *For every $s \in \mathcal{J}$ and $i, k \in N$,*

$$\begin{cases} b_{s,i} \leq V_i(c_{s,k} \cup A_{s,k}) \leq b_{s,i} + \eta_i, & k \notin F_{s,i}, \\ V_i(c_{s,k} \cup A_{s,k}) \leq b_{s,i} - \Gamma_i + \eta_i, & k \in F_{s,i}. \end{cases}$$

Proof. If $k \notin F_{s,i}$, then $d_{s,i,k} \leq \epsilon_i$. By [Lemma 4.14](#),

$$d_{s,i,k} \leq V_i(A_{s,k}) \leq \eta_i.$$

Since $V_i(c_{s,k}) = b_{s,i} - d_{s,i,k}$, the lower bound follows. Because snapshot s is envy-free, $V_i(c_{s,k}) \leq b_{s,i}$, so the upper bound follows as well.

If $k \in F_{s,i}$, then $d_{s,i,k} > \epsilon_i$. Since [Lemma 4.7](#) rules out bonuses strictly between ϵ_i and Γ_i , we have $d_{s,i,k} \geq \Gamma_i$. Together with $V_i(A_{s,k}) \leq \eta_i$, this gives

$$V_i(c_{s,k} \cup A_{s,k}) \leq b_{s,i} - \Gamma_i + \eta_i. \quad \square$$

To combine these bounds across all snapshots in $\mathcal{J}$, we group together the pieces assigned to each agent by the matchings. For every agent h , let D_h be the union of the snapshot pieces assigned to h , and let A_h be the union of their additions. For every agent i , let B_i be her total value for her own snapshot pieces:

$$D_h = \bigcup_{s \in \mathcal{J}} c_{s, \mu_s(h)}, \quad A_h = \bigcup_{s \in \mathcal{J}} A_{s, \mu_s(h)}, \quad B_i = \sum_{s \in \mathcal{J}} b_{s,i}.$$

Because every snapshot is envy-free, the two preceding lemmas give the following bounds.

Lemma 4.16. *For every $i, h \in N$, we have $V_i(D_h) \leq B_i$ and $V_i(A_h) \leq Q\eta_i$. Moreover:*

(i) if $\mathbf{F}_i = \mathbf{F}_h$, then $V_i(D_h \cup A_h) \geq B_i$;

(ii) if $\mathbf{F}_i \not\subseteq \mathbf{F}_h$, then $V_i(D_h \cup A_h) \leq B_i - \Gamma_i + Q\eta_i$.

Proof. Because every snapshot in $\mathcal{J}$ is envy-free, agent i values the snapshot piece assigned to h no more than her own piece in that snapshot. Summing over the snapshots gives $V_i(D_h) \leq B_i$. By definition, A_h contains one addition from each of the Q snapshots in $\mathcal{J}$. Each addition is worth at most η_i to agent i , so $V_i(A_h) \leq Q\eta_i$.

For (i), suppose $\mathbf{F}_i = \mathbf{F}_h$. For every $s \in \mathcal{J}$, the matching uses the edge $h \rightarrow \mu_s(h)$, so $\mu_s(h) \notin F_{s,h} = F_{s,i}$. By [Lemma 4.15](#),

$$V_i(c_{s, \mu_s(h)} \cup A_{s, \mu_s(h)}) \geq b_{s,i}.$$

Summing over all $s \in \mathcal{J}$ gives $V_i(D_h \cup A_h) \geq B_i$.

For (ii), suppose $F_i \not\leq F_h$. By [Lemma 4.12](#), some snapshot $s \in \mathcal{J}$ satisfies $k = \mu_s(h) \in F_{s,i}$. For this snapshot, [Lemma 4.15](#) gives

$$V_i(c_{s,k} \cup A_{s,k}) \leq b_{s,i} - \Gamma_i + \eta_i.$$

For every $t \in \mathcal{J}$ other than s , envy-freeness of snapshot t and the bound $V_i(A_{t,\mu_t(h)}) \leq \eta_i$ give

$$V_i(c_{t,\mu_t(h)} \cup A_{t,\mu_t(h)}) \leq b_{t,i} + \eta_i.$$

Summing these bounds gives

$$V_i(D_h \cup A_h) \leq B_i - \Gamma_i + \eta_i + (Q - 1)\eta_i = B_i - \Gamma_i + Q\eta_i. \quad \square$$

At this point, the additions make a piece good enough for an agent who does not mark its label, while keeping it substantially worse for any agent who does mark its label. The rank pieces provide more cake for classes of higher rank than for classes of lower rank.

Agents in the same profile class. Agents in the same profile class may still envy one another. For every agent h , we concatenate P_h , A_h , and D_h , in this order, to form

$$W_h = [P_h, A_h, D_h].$$

We now fix a profile class E and use SUBCORE to redistribute the cake prepared for its agents. For every agent $i \in E$, set her required minimum to

$$q_i = B_i + \max\{0, L(\ell_E - 1/2)\}V_i(R^*),$$

i.e., the first term is agent i 's value for her own pieces in the snapshots from $\mathcal{J}$, and the second is a lower bound on her value for every P_h with $h \in E$.

Lemma 4.17. (i) *The pieces $(W_h)_{h \in N}$ are pairwise disjoint.*

(ii) *For every $i, h \in E$, we have $V_i(W_h) \geq q_i$.*

(iii) *If we call SUBCORE on the agents in E , the pieces $(W_h)_{h \in E}$, and one empty piece, using q_i as agent i 's required minimum, then it produces an envy-free allocation. Every agent receives a suffix of a distinct W_h worth at least her required minimum. The empty piece remains unallocated, and a suffix is allocated from every W_h .*

(iv) *For every $h \in E$, the suffix allocated from W_h contains all of D_h . Consequently, the part of W_h left unallocated is a prefix contained in $P_h \cup A_h$.*

Proof. For (i), [Lemma 4.7](#) shows that the snapshot allocations are pairwise disjoint and lie outside R^* . For every $s \in \mathcal{J}$, the perfect matching assigns each label exactly once. Thus, each $c_{s,k}$ belongs to exactly one D_h , and the corresponding $A_{s,k}$ belongs to exactly one A_h . By [Lemma 4.14](#), all additions $A_{s,k}$ and rank pieces P_h are pairwise disjoint subsets of R^* . Hence, the pieces W_h are pairwise disjoint.

For (ii), fix agents $i, h \in E$. Since they have the same profile, [Lemma 4.16](#) gives $V_i(D_h \cup A_h) \geq B_i$. They also have the same rank, so $\ell_h = \ell_E$, and [Lemma 4.14](#) gives $V_i(P_h) \geq \max\{0, L(\ell_E - 1/2)\}V_i(R^*)$. Therefore,

$$\begin{aligned} V_i(W_h) &= V_i(P_h) + V_i(A_h \cup D_h) \\ &\geq B_i + \max\{0, L(\ell_E - 1/2)\}V_i(R^*) \\ &= q_i. \end{aligned}$$

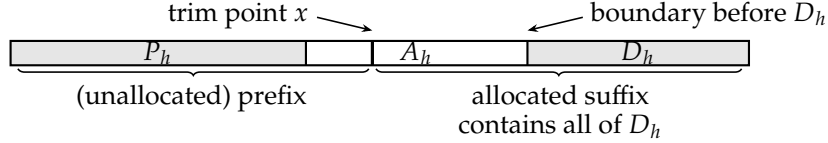


Figure 3: Every trim occurs no later than the boundary before D_h , so the allocated suffix contains all of D_h .

For (iii), (ii) shows that every agent $i \in E$ values every W_h at least q_i , so we may apply [Proposition 4.5](#). Therefore, SUBCORE gives every agent a suffix of a distinct W_h worth at least her required minimum and leaves the empty piece unallocated. There are $|E|$ agents and $|E|$ pieces W_h , so a suffix is allocated from every W_h .

For (iv), consider any trim. Suppose SUBCORE asks agent i to trim W_h , or one of its suffixes, to value q . Agent i 's required minimum starts at q_i and only increases, so $q \geq q_i$. By [Lemma 4.16](#),

$$V_i(D_h) \leq B_i \leq q_i \leq q.$$

If $V_i(D_h) < q$, then a suffix beginning at or after the boundary before D_h is worth less than q , so the trim occurs before that boundary. If $V_i(D_h) = q$, that boundary gives a suffix worth q , and the rule in [Section C.2](#) chooses it or an earlier boundary. Thus, every trim occurs no later than the boundary before D_h , as shown in [Figure 3](#). Every allocated suffix therefore contains D_h , and the part left unallocated is a prefix contained in $P_h \cup A_h$. $\square$

The partial allocation. We have gathered everything we need. For agents in different profile classes, [Lemmas 4.12](#) to [4.16](#) give us the additional cake needed to ensure that each agent values her own share more than the share of anyone in another class. For agents in the same profile class, [Lemma 4.17](#) gives us a way to redistribute their cake so that they no longer envy one another.

We apply [Lemma 4.17](#) to every profile class and combine the resulting allocations into Z . By [Lemma 4.17](#), Z is envy-free within each profile class and contains all snapshot pieces from $\mathcal{J}$. We use the identity assignment for every other snapshot and finally define

$$X = H \oplus \text{Id}(S \setminus \mathcal{J}) \oplus Z,$$

the partial allocation we seek and on which we build the recursion. (Recall that H comes from [Lemma 4.7](#).) We now prove that X is well defined and envy-free.

Lemma 4.18. *For every agent h , let Y_h be the prefix of W_h left unallocated by the SUBCORE call for h 's profile class. The allocation X is well defined: no cake is allocated twice, and its residue is*

$$R_1 = \left(R^* \setminus \bigcup_{h \in N} (P_h \cup A_h) \right) \cup \bigcup_{h \in N} Y_h \subseteq R^*.$$

Proof. By [Lemma 4.17](#) (i), the pieces W_h are pairwise disjoint, so the calls for different profile classes use disjoint cake. By [Lemma 4.17](#) (iii), the call for a profile class E assigns a suffix from every W_h with $h \in E$. Since the profile classes partition N , one suffix is assigned from every W_h . By [Lemma 4.17](#) (iv), this suffix contains all of D_h , so its unused prefix Y_h lies in $P_h \cup A_h$.

By [Lemma 4.14](#), the sets $P_h \cup A_h$, over all $h \in N$, are pairwise disjoint subsets of R^* . Removing these sets from R^* and returning the unused prefixes gives

$$R_1 = \left(R^* \setminus \bigcup_{h \in N} (P_h \cup A_h) \right) \cup \bigcup_{h \in N} Y_h.$$

Since $Y_h \subseteq P_h \cup A_h$ for every h , we also have $R_1 \subseteq R^*$. $\square$

It remains to prove that X is envy-free.

Lemma 4.19. *Assume every graph G_s with $s \in \mathcal{P}$ is strongly connected. Then X is envy-free and, for every pair (i, j) in different profile classes,*

$$\Delta_{ij}(X) \geq \eta_i = \frac{V_i(R^*)}{4n(Q+1)Q^2}.$$

Proof. Fix agents i, j in different profile classes. By Lemma 4.17 (iii), agent j receives a suffix of W_h for some h in her profile class. Hence $F_h = F_j$ and $\ell_h = \ell_j$. By definition, $LV_i(R^*) = 4(Q+1)\eta_i$ and $\Gamma_i = 4n(Q+1)\eta_i$.

Since i and j belong to different profile classes, either $F_i < F_j$ or $F_i \not\leq F_j$.

Suppose first $F_i < F_j$. By Lemma 4.13, $\ell_i \geq \ell_j + 1$. By the definition of q_i and Lemma 4.17 (iii),

$$V_i(Z_i) \geq B_i + L(\ell_i - \frac{1}{2})V_i(R^*).$$

Because every snapshot is envy-free, $V_i(D_h) \leq B_i$. By Lemmas 4.14 and 4.16, we also have

$$V_i(A_h) \leq Q\eta_i, \quad V_i(P_h) \leq L\ell_j V_i(R^*).$$

Because $Z_j \subseteq W_h$, we further have

$$V_i(Z_j) \leq B_i + Q\eta_i + L\ell_j V_i(R^*).$$

Together,

$$\begin{aligned} \Delta_{ij}(Z) &\geq L \left(\ell_i - \ell_j - \frac{1}{2} \right) V_i(R^*) - Q\eta_i \geq \frac{L}{2} V_i(R^*) - Q\eta_i \\ &= [2(Q+1) - Q] \eta_i = (Q+2)\eta_i \\ &\geq \eta_i. \end{aligned}$$

Now suppose $F_i \not\leq F_j$. By Lemma 4.17 (iii), $V_i(Z_i) \geq q_i \geq B_i$. Because $F_h = F_j$, we also have $F_i \not\leq F_h$. By Lemma 4.16, we have

$$V_i(D_h \cup A_h) \leq B_i - \Gamma_i + Q\eta_i.$$

In addition, Lemmas 4.13 and 4.14 give

$$V_i(P_h) \leq L(n-1)V_i(R^*).$$

Since $Z_j \subseteq W_h$, these bounds give

$$V_i(Z_j) \leq B_i - \Gamma_i + Q\eta_i + L(n-1)V_i(R^*).$$

Together,

$$\begin{aligned} \Delta_{ij}(Z) &\geq \Gamma_i - Q\eta_i - L(n-1)V_i(R^*) \\ &= [4n(Q+1) - Q - 4(n-1)(Q+1)] \eta_i \\ &= (3Q+4)\eta_i \geq \eta_i. \end{aligned}$$

Since i and j were arbitrary, the two estimates give $\Delta_{ij}(Z) \geq \eta_i$ whenever i and j belong to different profile classes. By Lemma 4.17, Z is envy-free within each profile class. Hence Z is envy-free. By Lemma 4.7, H and every snapshot in $\mathcal{S} \setminus \mathcal{J}$ are also envy-free. By Lemma 4.18, these allocations and Z use pairwise disjoint cake. Thus, Lemma 4.1 (i) gives $\Delta_{ij}(X) \geq \Delta_{ij}(Z)$ for all agents i, j , so X is envy-free and the proof is complete. $\square$

4.6 Recursion and Query Complexity

By [Lemmas 4.18](#) and [4.19](#), we now have an envy-free partial allocation X with residue $R_1 \subseteq R^*$. Moreover, [Lemma 4.19](#) bounds how much more each agent values her own share than the share of anyone in another profile class. We now use this bound to complete the recursion and then count the queries used by the protocol.

[Lemma 4.9](#) guarantees that at least two profile classes remain, so we choose one and call it E_* . To make every agent outside E_* dominate its members, we reduce her value for the residue to at most η_i . For each $i \in N \setminus E_*$ with $V_i(R_1) > 0$, we run a prescribed sequence of at most $\left\lceil \log_{n/(n-2)}(4n(Q+1)Q^2) \right\rceil$ [CORE](#) calls with i as the cutter, successively on the current residue. Let R_2 be the final residue after all these sequences, and let X' be obtained by adding every allocation returned by these calls to X . We recurse on E_* and return

$$X' \oplus \text{MAIN}(E_*, R_2).$$

Proposition 4.20. *Suppose every recursive call on fewer than n agents returns a complete envy-free allocation. Then $\text{MAIN}(N, C)$ returns a complete envy-free allocation, either without recursion or after one recursive call on a nonempty proper subset of agents.*

Proof. If every agent values R^* at zero, we give R^* to one agent and combine it with H and all snapshots under the identity assignment. By [Lemmas 4.1](#) and [4.7](#), this gives a complete envy-free allocation.

If instead some graph is not strongly connected, [Lemma 4.10](#) reduces the problem to one recursive call on a nonempty proper subset of agents. That call returns a complete envy-free allocation by induction.

We may therefore assume that every graph is strongly connected. In this case, [Lemma 4.9](#) gives at least two profile classes, so E_* is nonempty and proper. By [Lemmas 4.18](#) and [4.19](#), the allocation X is envy-free, has residue $R_1 \subseteq R^*$, and satisfies $\Delta_{ij}(X) \geq \eta_i$ whenever i and j belong to different profile classes.

Fix $i \in N \setminus E_*$ with $V_i(R_1) > 0$. When her prescribed sequence begins, the current residue is contained in R_1 , and all later calls only shrink it. Hence [Proposition 4.3](#) (iii) gives

$$V_i(R_2) \leq \frac{V_i(R_1)}{4n(Q+1)Q^2} \leq \frac{V_i(R^*)}{4n(Q+1)Q^2} = \eta_i.$$

Because every added [CORE](#) allocation is envy-free, [Lemma 4.1](#) (i) shows that X' remains envy-free and that $\Delta_{ij}(X') \geq \Delta_{ij}(X)$ for every pair of agents. Consequently, for every $j \in E_*$,

$$\Delta_{ij}(X') \geq \Delta_{ij}(X) \geq \eta_i \geq V_i(R_2).$$

Thus, i dominates every member of E_* . If $V_i(R_1) = 0$, then $V_i(R_2) = 0$, so envy-freeness of X' gives the same conclusion. Hence every agent outside E_* dominates every agent within it.

By induction, $\text{MAIN}(E_*, R_2)$ returns a complete envy-free allocation of R_2 . We may therefore combine it with X' using [Corollary 4.2](#). $\square$

It remains to count the queries.

Proposition 4.21. *Let $n \geq 4$. There are fixed constants $K, d > 0$ such that a call $\text{MAIN}(N, C)$ with $|N| = n$ uses at most $\Lambda_n = Kn^d 2^n$ queries, not counting those made by its recursive call. The sequence (Λ_n) is nondecreasing.*

Proof. Pre-processing uses $n^{O(1)}2^n$ queries by Lemma 4.7. After pre-processing, queries are used only for the prescribed CORE sequences, for cutting the additions and rank pieces, and for the SUBCORE calls within profile classes.

At most n prescribed sequences are used, each containing $O(n \log n)$ CORE calls. The simultaneous cutting step uses polynomially many queries because it produces polynomially many pieces and $1/\delta$ is polynomial in n . We also invoke SUBCORE at most once for each profile class.

Consequently, only polynomially many CORE and SUBCORE calls are made after pre-processing, each involving at most n agents. By Propositions C.1 and 4.4, these calls use $n^{O(1)}2^n$ queries in total. Combining these bounds, we can choose fixed constants $K, d > 0$ such that $\Lambda_n = Kn^d 2^n$ bounds all queries made outside recursion. Since $n^d 2^n$ increases with n , the sequence (Λ_n) is nondecreasing. $\square$

We are finally ready to prove the main theorem. Here, we restate it as a refresher.

Theorem 1.1 (restated). *For every $n \geq 1$ and every collection of nonnegative, additive, nonatomic valuations, there is a deterministic bounded Robertson–Webb protocol that returns a complete envy-free allocation using at most $n^{O(1)}2^n$ queries.*

Proof. We prove correctness by induction on n . The protocols in Section 4.3 handle $n \leq 3$, and Proposition 4.20 gives the inductive step for $n \geq 4$. We make every choice deterministic using the fixed orders and tie-breaking rules described in Section C.2.

To count the queries, let $T(n)$ be the largest number of queries used by a call with n agents, including its recursive calls, and set $\bar{T}(n) = \max_{1 \leq m \leq n} T(m)$. By Propositions 4.20 and 4.21, every call uses at most Λ_n queries outside recursion and makes at most one recursive call on fewer agents. Since (Λ_n) is nondecreasing,

$$\bar{T}(n) \leq \bar{T}(n-1) + \Lambda_n \leq \bar{T}(3) + \sum_{k=4}^n \Lambda_k \leq \bar{T}(3) + n\Lambda_n.$$

Therefore $T(n) \leq n^{O(1)}2^n$, as claimed. $\square$

5 Discussion and Concluding Remarks

Our protocol gives the first single-exponential upper bound for finding a complete envy-free allocation. Yet the $\Omega(n^2)$ lower bound of Procaccia [26] leaves a wide gap. To better understand the complexity of our protocol, we consider two questions. First, how much computation is needed between queries? Second and more importantly, where does the exponential dependence in our query bound come from?

For the first question, the main issue appears in Lemma 4.6, where the proof uses an exhaustive search to assign the cells. Here, we observe that this search is not necessary. It turns out that we can instead assign each cell randomly according to the probabilities used in the proof. The same concentration argument shows that the resulting assignment succeeds with high probability. Because we already know the value of every cell, we can check whether an assignment succeeds without making any further queries. Repeating this process until it succeeds therefore gives a simple randomized procedure. We also note that we can remove the randomness using the standard method of conditional expectations [27, 31], by choosing the cells one at a time while preserving the same guarantee. Hence, we can find a valid assignment deterministically using polynomially many arithmetic operations in the number of cells.

For the second question, our proof gives a clear answer. We maintain only polynomially many partial allocations, make only polynomially many calls to `CORE` and `SUBCORE`, and use only polynomially many queries outside these calls. Thus, up to polynomial factors, the full protocol costs no more than one call to `CORE` or `SUBCORE` involving n agents. In particular, combining the partial allocations does not add another exponential factor.

Hence, the remaining exponential cost comes from calls to `CORE` and `SUBCORE` involving $\Theta(n)$ agents. Naturally, this suggests two ways forward. One is to improve the query bounds for these subroutines, and the other is to avoid calling them with so many agents. Indeed, if every call involved $o(n)$ agents, then our protocol would use $2^{o(n)}$ queries. Can either approach yield a subexponential or even polynomial protocol? Or is the quadratic lower bound tight? We believe that answering these questions will deepen our understanding of the computational limits of fair division, and we hope our approach provides a useful starting point.

Acknowledgments

We thank Chaoran Yu for helpful feedback on a preliminary draft of this manuscript.

References

- [1] Georgios Amanatidis, George Christodoulou, John Fearnley, Evangelos Markakis, Christos-Alexandros Psomas, and Eftychia Vakaliou. An improved envy-free cake cutting protocol for four agents. In *11th International Symposium on Algorithmic Game Theory (SAGT)*, pages 87–99, 2018. doi: 10.1007/978-3-319-99660-8_9.
- [2] Eshwar Ram Arunachaleswaran, Siddharth Barman, Rachitesh Kumar, and Nidhi Rathi. Fair and efficient cake division with connected pieces. In *15th International Conference on Web and Internet Economics (WINE)*, pages 57–70, 2019. doi: 10.1007/978-3-030-35389-6_5.
- [3] Haris Aziz and Simon Mackenzie. A discrete and bounded envy-free cake cutting protocol for any number of agents. In *57th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 416–427, 2016. doi: 10.1109/focs.2016.52. Extended version: arXiv:1604.03655v12.
- [4] Haris Aziz and Simon Mackenzie. A discrete and bounded envy-free cake cutting protocol for four agents. In *48th Annual ACM Symposium on Theory of Computing (STOC)*, pages 454–464, 2016. doi: 10.1145/2897518.2897522.
- [5] Haris Aziz and Chun Ye. Cake cutting algorithms for piecewise constant and piecewise uniform valuations. In *10th International Conference on Web and Internet Economics (WINE)*, pages 1–14, 2014. doi: 10.1007/978-3-319-13129-0_1.
- [6] Siddharth Barman and Pooja Kulkarni. Approximation algorithms for envy-free cake division with connected pieces. In *50th International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 16:1–16:19, 2023. doi: 10.4230/lipics.icalp.2023.16.
- [7] Siddharth Barman and Nidhi Rathi. Fair cake division under monotone likelihood ratios. *Math. Oper. Res.*, 47(3):1875–1903, 2022. doi: 10.1287/moor.2021.1192.

- [8] Xiaohui Bei, Youming Qiao, and Shengyu Zhang. Networked fairness in cake cutting. In *26th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 3632–3638, 2017. doi: 10.24963/ijcai.2017/508.
- [9] Noah Bertram, Alex Levinson, and Justin Hsu. Cutting the cake: A language for fair division. *Proc. ACM Program. Lang.*, 7(PLDI):1779–1800, 2023. doi: 10.1145/3591293.
- [10] Noah Bertram, Tean Lai, and Justin Hsu. Verifying cake-cutting, faster. In *36th International Conference on Computer Aided Verification (CAV)*, pages 109–129, 2024. doi: 10.1007/978-3-031-65630-9_6.
- [11] Steven J. Brams and Alan D. Taylor. An envy-free cake division protocol. *Amer. Math. Monthly*, 102(1):9–18, 1995. doi: 10.2307/2974850.
- [12] Steven J. Brams and Alan D. Taylor. *Fair division - from cake-cutting to dispute resolution*. Cambridge University Press, 1996. ISBN 978-0-521-55644-6.
- [13] Simina Brânzei. A note on envy-free cake cutting with polynomial valuations. *Inf. Process. Lett.*, 115(2):93–95, 2015. doi: 10.1016/j.ipl.2014.07.005.
- [14] Simina Brânzei and Noam Nisan. The query complexity of cake cutting. In *36th Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2022. doi: 10.52202/068431-2747.
- [15] Guillaume Chèze. Envy-free cake cutting: a polynomial number of queries with high probability. *Soc. Choice Welf.*, 67(1):1–20, 2026. doi: 10.1007/s00355-025-01633-7.
- [16] Xiaotie Deng, Qi Qi, and Amin Saberi. Algorithmic solutions for envy-free cake cutting. *Oper. Res.*, 60(6):1461–1476, 2012. doi: 10.1287/opre.1120.1116.
- [17] Ruiquan Gao, Mohammad Roghani, Aviad Rubinstein, and Amin Saberi. Hardness of approximate sperner and applications to envy-free cake cutting. In *65th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 1294–1331, 2024. doi: 10.1109/focs61266.2024.00085.
- [18] Ganesh Ghalme, Xin Huang, Yuka Machino, and Nidhi Rathi. A discrete and bounded locally envy-free cake cutting protocol on trees. In *19th International Conference on Web and Internet Economics (WINE)*, pages 310–328, 2023. doi: 10.1007/978-3-031-48974-7_18.
- [19] Paul W. Goldberg, Alexandros Hollender, and Warut Suksompong. Contiguous cake cutting: Hardness results and approximation algorithms. *J. Artif. Intell. Res.*, 69:109–141, 2020. doi: 10.1613/jair.1.12222.
- [20] Alexandros Hollender and Aviad Rubinstein. Envy-free cake-cutting for four agents. *J. ACM*, 72(5):34:1–34:54, 2025. doi: 10.1145/3765615.
- [21] Alexandros Hollender, Gilbert Maystre, and Kilian Risse. On cutting cakes and crossing curves. *CoRR*, abs/2606.13980, 2026. doi: 10.48550/arxiv.2606.13980.
- [22] David Kurokawa, John K. Lai, and Ariel D. Procaccia. How to cut a cake before the party ends. In *27th AAAI Conference on Artificial Intelligence (AAAI)*, pages 555–561, 2013. doi: 10.1609/aaai.v27i1.8629.

- [23] Martin Mariusz Lester. Cutting the cake into crumbs: Verifying envy-free cake-cutting protocols using bounded integer arithmetic. In *26th International Symposium on Practical Aspects of Declarative Languages (PADL)*, pages 100–115, 2024. doi: 10.1007/978-3-031-52038-9_7.
- [24] Arnav Mehra and Alexandros Psomas. On hierarchies of fairness notions in cake cutting: From proportionality to super envy-freeness. In *39th Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2025. doi: 10.52202/085713-1813.
- [25] Oleg Pikhurko. On envy-free cake division. *Amer. Math. Monthly*, 107(8):736–738, 2000. URL <http://www.jstor.org/stable/2695471>.
- [26] Ariel D. Procaccia. Thou shalt covet thy neighbor’s cake. In *21st International Joint Conference on Artificial Intelligence (IJCAI)*, pages 239–244, 2009. URL <http://ijcai.org/Proceedings/09/Papers/049.pdf>.
- [27] Prabhakar Raghavan. Probabilistic construction of deterministic algorithms: Approximating packing integer programs. *J. Comput. Syst. Sci.*, 37(2):130–143, 1988. doi: 10.1016/0022-0000(88)90003-7.
- [28] Jack M. Robertson and William A. Webb. *Cake-cutting algorithms - be fair if you can*. A K Peters, 1998. ISBN 978-1-56881-076-8.
- [29] Erel Segal-Halevi, Avinatan Hassidim, and Yonatan Aumann. Waste makes haste: Bounded time protocols for envy-free cake cutting with free disposal. In *14th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 901–908, 2015. URL <http://dl.acm.org/citation.cfm?id=2773268>.
- [30] Georgy Sokolov. An exponential envy-free cake cutting protocol for n agents. *CoRR*, abs/2306.03854, 2023. doi: 10.48550/arxiv.2306.03854.
- [31] Anand Srivastav and Peter Stangier. Algorithmic chernoff-hoeffding inequalities in integer programming. *Random Struct. Algorithms*, 8(1):27–58, 1996.
- [32] Hugo Steinhaus. The problem of fair division. *Econometrica*, 16(1):101–104, 1948. doi: 10.2307/1914289.
- [33] Walter Stromquist. How to cut a cake fairly. *Amer. Math. Monthly*, 87(8):640–644, 1980.
- [34] Walter Stromquist. Envy-free cake divisions cannot be found by finite protocols. *Electron. J. Comb.*, 15(1), 2008. URL http://www.combinatorics.org/Volume_15/Abstracts/v15i1r11.html.
- [35] Francis Edward Su. Rental harmony: Sperner’s lemma in fair division. *Amer. Math. Monthly*, 106(10):930–942, 1999.
- [36] Chenhao Wang and Xiaoying Wu. Cake cutting with single-peaked valuations. In *13th International Conference on Combinatorial Optimization and Applications (COCO)*, pages 507–516, 2019. doi: 10.1007/978-3-030-36412-0_41.

A AI Usage Disclosure

We used ChatGPT 5.5 Pro/5.6 Sol extensively during the *early* brainstorming stage, and we briefly describe its role here. GPT’s most important contribution was the suggestion that the rather computationally heavy GoLeft protocol of Aziz and Mackenzie could be replaced by a much more efficient approach. It developed this suggestion into a complete protocol using $2^{2n(n-1)+O(n \log n)}$ Robertson–Webb queries. This GPT-generated protocol provided the conceptual backbone for our work and contained prototypes of several structural ideas used in our paper.

From this starting point, we refined and adapted GPT’s structural ideas to the needs of our protocol. In our construction, these refined ideas eventually take the form of marks, witnesses, profiles, and ranks, which we use to determine how snapshot pieces are chosen and redistributed.

Beyond this structural work, and independent of LLM assistance, we studied where the quadratic exponent in GPT’s bound came from, and we found the source to be twofold. First, GPT’s construction kept exponentially many snapshots in order to find sufficiently many with exactly the same marks (roughly speaking, this is the precondition of [Lemma 4.12](#)). Second, it constructed the additions one at a time, and an unsuccessful attempt could require returning the cake and repeating this work across the stored snapshots. Together, these choices made the cost grow like the square of the number of snapshots.

To address these inefficiencies, we introduced two further ideas. First, we begin with only polynomially many snapshots and carefully prune them ([Section 4.4](#)). We show that the resulting collection still contains enough witnesses to reserve a different snapshot for every assignment we need ([Lemmas 4.9](#) and [4.12](#)). Second, to avoid constructing the additions one at a time, we developed a “simultaneous cutting” lemma ([Lemma 4.6](#)), which constructs all additions and rank pieces together using polynomially many queries ([Lemma 4.14](#)). These ideas further improve GPT’s $2^{2n(n-1)+O(n \log n)}$ bound to $2^{O(n)}$ and allow us to streamline the algorithm (in particular, the recursion logic becomes much easier to describe).

While preparing the final manuscript, we used GPT 5.6 to help identify part of additional related work and to generate the full pseudocode in [Appendix E](#). We developed the full protocol and proofs and wrote the remainder (and hence, almost all) of the paper, and we take full responsibility for its correctness and integrity.

B Additional Related Work

Here we expand on the related work mentioned briefly in [Section 1](#).

If every agent must receive one interval, Stromquist proved that no finite protocol can guarantee a complete envy-free allocation for three or more agents [\[34\]](#). For additive valuations, work has therefore studied query bounds, approximation algorithms, and welfare optimization for connected pieces [\[2, 6, 14, 19\]](#). In a broader model of continuous utilities over connected pieces, Deng, Qi, and Saberi gave an efficient approximation algorithm for three monotone agents, and Hollender and Rubinstein extended this positive result to four agents while proving hardness without monotonicity [\[16, 20\]](#). Hollender, Maystre, and Risse have since shown intractability for three agents in the general continuous-utility model [\[21\]](#). For a fixed number of possibly disconnected pieces, Gao et al. prove PPAD-hardness in another nonadditive model, even when the output need only make three agents approximately envy-free [\[17\]](#). These models allow utilities more general than the additive valuations in our theorem. With free disposal, bounded envy-free protocols can instead give every agent a connected piece of positive value while allowing some cake to remain unallocated [\[29\]](#).

Structural restrictions on additive valuations lead to stronger algorithms. Kurokawa, Lai, and Procaccia give a Robertson–Webb protocol using $O(n^6 k \log k)$ queries when the agents’ piecewise-linear densities have at most k breakpoints in total, and Brânzei gives an envy-free protocol for polynomial value densities whose query bound also depends on the maximum degree [13, 22]. In a direct-revelation model, Aziz and Ye give polynomial algorithms for piecewise-constant valuations [5]. For single-peaked valuations, an exact envy-free allocation can be found using a linear number of queries, while the monotone-likelihood-ratio condition yields efficient connected allocations with envy-freeness to arbitrary precision [7, 36]. These guarantees depend on structural restrictions absent from our unrestricted model.

The bounded protocol for four agents that preceded AM’s general protocol was later simplified, and its query bound was reduced by a factor of 3.4 [1]. Bei, Qiao, and Zhang introduced local envy-freeness on a graph, and subsequent work gave bounded discrete protocols when the graph is a tree [8, 18]. Beyond worst-case analysis, Chèze proves that Webb’s protocol uses fewer than n^{12} queries with high probability under the uniform full-independence model [15]. Mehra and Psomas introduce hierarchies of fairness notions that connect proportionality, envy-freeness, and super envy-freeness and study their query complexity [24]. The complexity of modern protocols has also prompted automated verification tools, including the SLICE language, the CRUMBS approach based on bounded model checking, and a faster SLICE verification framework [9, 10, 23].

C Robertson–Webb Implementation Details

In this appendix, we explain how ordinary Robertson–Webb queries handle pieces with several interval components and record the deterministic conventions used by the protocol.

C.1 Queries on Ordered Pieces

We ignore overlaps at finitely many cut endpoints, which have value zero for every agent. In Robertson–Webb query complexity, exact arithmetic, comparisons, and finite computation on oracle answers are free, and we do not bound the number of bits needed to represent arbitrary real answers.

Whenever the protocol later divides a piece with several interval components, it stores those components in a fixed order, for instance from left to right. We call a piece with such an order an *ordered piece*. For disjoint pieces P, A, D , the notation $[P, A, D]$ denotes their union with the components of P first, followed by those of A and then those of D . A *prefix* ending at a point x contains everything through x in this order, while the corresponding *suffix* contains everything from x onward. For example, take

$$P = [0, 0.1] \cup [0.7, 0.8], \quad A = [0.2, 0.3], \quad D = [0.4, 0.5] \cup [0.9, 1].$$

Then $[P, A, D]$ orders its components as

$$[0, 0.1], [0.7, 0.8], [0.2, 0.3], [0.4, 0.5], [0.9, 1],$$

and splitting the component $[0.2, 0.3]$ at $x = 0.25$ gives the prefix $P \cup [0.2, 0.25]$ and the suffix $[0.25, 0.3] \cup D$.

Proposition C.1. *Let N be a set of agents. Queries to an ordered piece can be implemented with ordinary Robertson–Webb queries. Once we know how every agent in N values every interval component, we can evaluate the ordered piece without further queries. Asking for a prefix or suffix of a given value uses at most one ordinary cut query and $O(|N|)$ evaluation queries, regardless of the number of components.*

Proof. We maintain these values for every agent in the current call. At the top level, one evaluation query per agent records the value of the initial interval $[0, 1]$. If a later recursive call receives a subcake with several components, those components were cut earlier, so we already know how every agent values them. The value of any ordered piece is therefore the sum of the known values of its components.

Suppose agent c asks for a prefix worth a . Starting with the first component, we add c 's values until the sum reaches or exceeds a . If it reaches a at a component boundary, that boundary gives the desired prefix without a cut query. Otherwise, the prefix ends inside the next component, where one ordinary cut query locates its endpoint. A suffix is handled in the same way from the last component, or equivalently by cutting a prefix of the complementary value.

After a component is split, the amount requested in the cut query gives c 's value of one resulting interval, and subtraction gives her value of the other. Every other agent in N evaluates one of the two intervals once, after which subtraction again gives the value of the other. We therefore know all new component values and can maintain the same information throughout the protocol. $\square$

C.2 Deterministic Choices

Because the protocol is deterministic, we resolve every finite choice by a fixed rule. Inside the two subroutines, we use the AM rule for resolving ties [3, Sec. 3.1.1 and the proof of Lemma 4.13 in the extended version]; recursive calls made inside SUBCORE keep the same order on their remaining agents and pieces. If the cutter values the residue at zero while we are collecting snapshots, we divide it into one possibly empty piece for each active agent by a fixed rule; she values all of them equally. In the direct SUBCORE calls of Proposition 4.5, a tie between one of the pieces W_h and the extra empty piece is resolved in favor of W_h .

After trimming an ordered piece, we keep its remaining interval components in the same order. Whenever we need a prefix or suffix of a given value, we examine the component boundaries from the beginning of this order and use the first one that gives the requested value. If none does, one ordinary cut query locates the cut point inside a component. Thus, when an agent values the whole ordered piece at zero, the convention gives her the empty prefix and the full suffix.

D Omitted Proofs for Section 4

Omitted proof of Proposition 4.4. To describe the modification, we temporarily use AM's notation for SUBCORE. AM calls the value that an agent must receive at a given point her benchmark. Our variant makes one change to AM's SUBCORE: we skip line 12 of AM's Algorithm 2. In other words, once $|W| = m - 1$, AM makes one final recursive call on W , while we omit that call. If $|W| = m - 1$ as soon as the trims are made, then every contested piece has a different rightmost trimmer. For each contested piece, we give its rightmost trimmer the part beginning at her trim. If the while loop runs, we keep the tentative allocation obtained in its final iteration. In either case, we give the sole agent outside W her favorite uncontested piece.

We argue by the same induction as AM that our variant preserves the guarantees of SUBCORE. The base case is immediate for one agent. If the next agent's favorite piece is unallocated, she receives it whole. Otherwise, the agents place their trims and form the set W . If $|W| = m - 1$, the rightmost trims give the agents in W an envy-free allocation in which every agent meets her benchmark. If $|W| < m - 1$, every recursive call inside the while loop has fewer than m agents, so the induction hypothesis applies. AM's argument then shows that each iteration adds one agent to W while preserving an envy-free tentative allocation in which every agent meets her benchmark [3,

proof of Lemma 4.13 in the extended version]. Thus, when the loop ends, the agents in W receive parts of distinct contested pieces without envy and meet their benchmarks.

Every agent in W receives at least as much value as she assigns to her favorite uncontested piece, so she envies no uncontested piece. The remaining agent receives her favorite uncontested piece, which AM's argument shows is worth at least her input benchmark. Every contested part begins at or to the right of her trim, so she does not envy any agent in W . The resulting allocation is therefore envy-free and meets every benchmark. The agents receive cake from distinct input pieces, and the agent outside W receives a whole piece.

We now return to the terminology of our paper and count the queries. Set $S_0 = S_1 = 0$, and for $m \geq 2$ define

$$S_m = S_{m-1} + m(m-1) + \sum_{j=1}^{m-2} S_j.$$

We claim that SUBCORE on m agents makes at most S_m cut queries. Indeed, processing the first $m-1$ agents uses at most S_{m-1} cuts, and introducing the m th agent requires at most $m(m-1)$ trims. The recursive calls inside the while loop have distinct sizes in $\{1, \dots, m-2\}$. Thus, S_m bounds the number of cuts by induction. For $m \geq 3$, subtracting the recurrence for S_{m-1} gives

$$S_m = 2S_{m-1} + 2(m-1).$$

The same identity holds for $m = 2$, and $S_1 = 0$, so

$$S_m = 2^{m+1} - 2m - 2 = O(2^m).$$

By [Proposition C.1](#), each cut uses $O(m)$ evaluation queries to record how the agents value the two resulting parts. Thus, the call uses $O(mS_m) = O(m2^m)$ evaluation queries.

A CORE call first uses $n-1$ cuts to create n pieces that the cutter values equally, followed by $O(n^2)$ evaluations to record how the agents value them. It then invokes SUBCORE on $n-1$ agents, so the stated CORE bounds follow. $\square$

Omitted proof of [Proposition 4.5](#). We compare our call with the same variant of SUBCORE in which every initial required minimum is zero. Assume that the two calls have proceeded in the same way up to the point where they introduce the r th agent. At most $r-1$ of the pieces W_h have been tentatively assigned, so some whole W_h remains unassigned. For every agent j currently taking part, let f_j be her value for her favorite unassigned piece. Since every W_h is worth at least q_j , we have $f_j \geq q_j$. Therefore,

$$\max\{q_j, f_j\} = f_j = \max\{0, f_j\}.$$

The required minima do not affect any trim, and the two calls make the same choices. Their recursive calls receive the same inputs, so induction on the number of agents shows that the two calls have the same execution.

Because the two calls have the same execution, the correctness argument for [Proposition 4.4](#) gives an envy-free allocation in which each agent receives a suffix of a different input piece. We implement the corresponding trims using [Proposition C.1](#). Moreover, agent i 's benchmark starts at q_i and only increases. Therefore, every agent receives value at least q_i .

It remains to show that the empty piece stays unallocated. Indeed, whenever we assign a piece directly, some whole W_h remains available and is worth at least as much as the empty piece. If the agent values W_h and the empty piece equally, our rule chooses W_h . Thus, we never assign the empty piece directly. Since the recursive calls use only contested pieces, the empty piece never enters a recursive call either. Therefore, it remains unallocated. $\square$

Omitted proof of [Lemma 4.14](#). Every ratio $d_{s,i,k}/V_i(R^*)$ appearing in the maximum that defines $r_{s,k}$ is at most $L/[4n(Q+1)]$. Thus, every addition has requested size at most

$$r_{s,k} + \delta \leq \frac{5L}{16n(Q+1)}.$$

There are nQ additions. By [Lemma 4.13](#), the total requested size of all additions and all P_h with $\ell_h > 0$ is at most

$$\begin{aligned} \frac{5QL}{16(Q+1)} + L \sum_h \ell_h + n\delta &\leq \frac{5QL}{16(Q+1)} + LQ + \frac{L}{16(Q+1)} \\ &= LQ + \frac{(5Q+1)L}{16(Q+1)} \\ &< 1. \end{aligned}$$

Therefore, [Lemma 4.6](#) applies.

Suppose that $d_{s,i,k} \leq \epsilon_i$ and $V_i(R^*) > 0$. The lower bound in [Lemma 4.6](#) gives

$$V_i(A_{s,k}) \geq r_{s,k} V_i(R^*) \geq d_{s,i,k}.$$

If $V_i(R^*) = 0$, then the small bonus is zero and the same inequality is immediate. For every agent,

$$V_i(A_{s,k}) \leq (r_{s,k} + 2\delta) V_i(R^*) \leq \frac{3L}{8n(Q+1)} V_i(R^*) \leq \eta_i.$$

If $\ell_h > 0$, we asked for P_h to contain approximately the fraction $L(\ell_h - \frac{1}{2}) + \delta$ of the residue for every agent. Thus, [Lemma 4.6](#) gives

$$L(\ell_h - \frac{1}{2}) V_i(R^*) \leq V_i(P_h) \leq [L(\ell_h - \frac{1}{2}) + 2\delta] V_i(R^*) \leq L\ell_h V_i(R^*),$$

because $2\delta \leq L/2$. If $\ell_h = 0$, then $P_h = \emptyset$, which satisfies the same bounds. Finally, the number of pieces and $1/\delta$ are polynomial in n , so the query bound follows from [Lemma 4.6](#). $\square$

E Full Pseudocode

This appendix gives pseudocode for the protocol in [Sections 3](#) and [4](#). We use the base protocols from [Section 4.3](#) and the `CORE` and `SUBCORE` subroutines from [Section 4.1](#). Whenever the pseudocode asks us to choose among finitely many options, we use the rule in [Section C.2](#). As in [Proposition 4.3](#) (iii), `Shrink`(N, U, K, c, D) repeatedly calls `CORE` on successive residues with agents N and cutter c . It makes at most D calls, stops if $V_c(U) = 0$, adds the resulting allocations to K , and returns the new allocation and residue.

Algorithm 1: MAIN(N, C)

Input: A nonempty active agent set N and a subcake C formed from finitely many intervals
Output: A complete envy-free allocation of C among N

Set $n \leftarrow |N|$
if $n \leq 3$ **then**
 return the allocation from [Section 4.3](#)

// stage 1: pre-processing
Set $Q \leftarrow n(n-1)$ and $(H, S, R^*, d) \leftarrow \text{Preprocess}(N, C)$
// stage 2: prune the snapshots
if $V_i(R^*) = 0$ for every $i \in N$ **then**
 return the allocation obtained from $H \oplus \text{Id}(S)$ by giving R^* to an agent

Set $\epsilon_i \leftarrow V_i(R^*)/[4n^2(Q+1)Q^2]$ for every $i \in N$, and set $F_{s,i} \leftarrow \{k \in N : d_{s,i,k} > \epsilon_i\}$ for every $s \in S$ and $i \in N$
Choose c with $V_c(R^*) > 0$ and set $\mathcal{P} \leftarrow \{(c, t) : 1 \leq t \leq Q^2\}$
while some pair (i, h) has between 1 and $Q-1$ witnesses in $\mathcal{P}$ **do**
 Choose such a pair and remove all of its witness snapshots from $\mathcal{P}$

For every $s \in \mathcal{P}$, form G_s whose edges are $i \rightarrow k$ with $k \notin F_{s,i}$
if some G_s is not strongly connected **then**
 Choose such an s and a sink strongly connected component S of G_s as in [Lemma 4.10](#)
 Set $Y \leftarrow H \oplus \text{Id}(S)$ and $U \leftarrow R^*$
 foreach $i \in S$ with $V_i(R^*) > 0$ **do**
 Set $(Y, U) \leftarrow \text{Shrink}(N, U, Y, i, \lceil \log_{n/(n-2)} Q^2 \rceil)$
 return $Y \oplus \text{MAIN}(N \setminus S, U)$

// stage 3: construct the partial allocation
Set $F_i \leftarrow (F_{s,i})_{s \in \mathcal{P}}$ and let $\mathcal{E}$ be the partition of N into profile classes
For every i, h , set $\mathcal{W}_{i,h} \leftarrow \{s \in \mathcal{P} : F_{s,i} \not\subseteq F_{s,h}\}$
Set $\mathcal{T} \leftarrow \{(i, h) : F_i \not\subseteq F_h\}$ and $\mathcal{J} \leftarrow \emptyset$
// reserve witnesses and choose matchings ([Lemma 4.12](#))
foreach $(i, h) \in \mathcal{T}$ **do**
 Choose $s \in \mathcal{W}_{i,h} \setminus \mathcal{J}$ and $k \in F_{s,i} \setminus F_{s,h}$
 Choose a perfect matching μ_s in G_s satisfying $\mu_s(h) = k$, and set $\mathcal{J} \leftarrow \mathcal{J} \cup \{s\}$

while $|\mathcal{J}| < Q$ **do**
 Choose $s \in \mathcal{P} \setminus \mathcal{J}$, set μ_s to the identity matching, and set $\mathcal{J} \leftarrow \mathcal{J} \cup \{s\}$

For every $E \in \mathcal{E}$, let ℓ_E be its rank as defined in [Section 4.5](#), and set $\ell_h \leftarrow \ell_E$ for every $h \in E$
Set $(Z, R_1) \leftarrow \text{Allocate}(N, R^*; \mathcal{J}, \mu; \mathcal{E}, \ell)$
Set $X \leftarrow H \oplus \text{Id}(S \setminus \mathcal{J}) \oplus Z$
Choose a profile class $E_* \subseteq N$, which exists by [Lemma 4.9](#), and set $U \leftarrow R_1$
foreach $i \in N \setminus E_*$ with $V_i(R_1) > 0$ **do**
 Set $(X, U) \leftarrow \text{Shrink}(N, U, X, i, \lceil \log_{n/(n-2)}(4n(Q+1)Q^2) \rceil)$

return $X \oplus \text{MAIN}(E_*, U)$

Algorithm 2: Preprocess(N, C)

Input: An active agent set N and its subcake C
Output: An envy-free partial allocation H , snapshots S , the final residue R^* , and the bonuses d

Set $n \leftarrow |N|$ and $Q \leftarrow n(n-1)$
Set $U \leftarrow C, S \leftarrow \emptyset$, and H to the empty allocation
// construct the snapshots
foreach $c \in N$ **do**
 for $t \leftarrow 1$ to Q^2 **do**
 Set $(X, U) \leftarrow \text{CORE}(N, U, c)$ and store X as snapshot $s = (c, t)$
 Write $c_{s,k}$ for its label- k piece for every k , and set $S \leftarrow S \cup \{s\}$

Record $V_i(c_{s,k})$ for every s, i, k
Set $b_{s,i} \leftarrow V_i(c_{s,i})$ for every s, i , and set $d_{s,i,k} \leftarrow b_{s,i} - V_i(c_{s,k})$ for every s, i, k
// eliminate the intermediate bonuses
Set $\Gamma_i \leftarrow V_i(U)/Q^2$, $\eta_i \leftarrow \Gamma_i/[4n(Q+1)]$, and $\epsilon_i \leftarrow \eta_i/n$ for every i
Set $D_{\text{mid}} \leftarrow \lceil \log_{n/(n-2)}(4n^2(Q+1)) \rceil$
while some agent i has an intermediate bonus **do**
 Choose such an i and set $(H, U) \leftarrow \text{Shrink}(N, U, H, i, D_{\text{mid}})$
 Recompute $\Gamma_r, \eta_r, \epsilon_r$ from the new U for every $r \in N$

Set $R^* \leftarrow U$ and **return** (H, S, R^*, d)

Algorithm 3: Allocate($N, R^*; \mathcal{J}, \mu; \mathcal{E}, \ell$)

Input: The active set N and residue R^* ; the snapshots $\mathcal{J}$ and their matchings μ ; the profile classes $\mathcal{E}$ and their ranks ℓ

Output: An allocation Z and the residue R_1

Set $n \leftarrow |N|$, $Q \leftarrow n(n-1)$, $L \leftarrow 1/(nQ^2)$, and $\delta \leftarrow L/[16n(Q+1)]$

Set $b_{s,i} \leftarrow V_i(c_{s,i})$ and $d_{s,i,k} \leftarrow b_{s,i} - V_i(c_{s,k})$ for every $s \in \mathcal{J}$ and $i, k \in N$

Set $\epsilon_i \leftarrow V_i(R^*)/[4n^2(Q+1)Q^2]$ for every $i \in N$

// cut the additions and rank pieces (Lemma 4.14)

For every $s \in \mathcal{J}$ and $k \in N$, set $r_{s,k} \leftarrow \max\{0, d_{s,i,k}/V_i(R^*) \text{ where } i \in N, V_i(R^*) > 0, d_{s,i,k} \leq \epsilon_i\}$

Set $I \leftarrow \{a_{s,k} : (s, k) \in \mathcal{J} \times N\} \cup \{p_h : \ell_h > 0\}$

Set $\lambda_{a_{s,k}} \leftarrow r_{s,k} + \delta$ for every $(s, k) \in \mathcal{J} \times N$ and $\lambda_{p_h} \leftarrow L(\ell_h - 1/2) + \delta$ for every h with $\ell_h > 0$

Apply Algorithm 4 to $N, R^*, I, (\lambda_j)_{j \in I}, \delta$, and denote its output by $(C_j)_{j \in I}$

Set $A_{s,k} \leftarrow C_{a_{s,k}}$ for every $(s, k) \in \mathcal{J} \times N$; set $P_h \leftarrow C_{p_h}$ when $\ell_h > 0$ and $P_h \leftarrow \emptyset$ otherwise

Set $D_h \leftarrow \bigcup_{s \in \mathcal{J}} c_{s, \mu_s(h)}$ and $A_h \leftarrow \bigcup_{s \in \mathcal{J}} A_{s, \mu_s(h)}$ for every h , and set $B_i \leftarrow \sum_{s \in \mathcal{J}} b_{s,i}$ for every i

// redistribute the cake within each profile class (Lemma 4.17)

foreach $E \in \mathcal{E}$ **do**

 Set $W_h \leftarrow [P_h, A_h, D_h]$ for $h \in E$ and $q_i \leftarrow B_i + \max\{0, L(\ell_E - 1/2)\}V_i(R^*)$ for $i \in E$

 Break any tie between a piece W_h and $\emptyset$ in favor of W_h , and set $Z^E \leftarrow \text{SubCore}(E, ((W_h)_{h \in E}, \emptyset), (q_i)_{i \in E})$

 Record every agent's value for the interval components created by each trim

 For every $h \in E$, let Y_h be the unused prefix of W_h

Set $Z \leftarrow \bigoplus_{E \in \mathcal{E}} Z^E$

Set $R_1 \leftarrow (R^* \setminus \bigcup_{h \in N} (P_h \cup A_h)) \cup \bigcup_{h \in N} Y_h$ as in Lemma 4.18

return (Z, R_1)

Algorithm 4: Procedure in Lemma 4.6

Input: An agent set N ; an ordered piece R , together with every agent's value for each interval that forms R ; a nonempty finite index set J ; numbers $\lambda_j \geq 0$ satisfying $\sum_{j \in J} \lambda_j \leq 1$; and $0 < \delta < 1$

Output: Pairwise disjoint pieces $(C_j)_{j \in J}$ satisfying $|V_i(C_j) - \lambda_j V_i(R)| \leq \delta V_i(R)$ for every $i \in N, j \in J$

Set $n \leftarrow |N|$, $m \leftarrow |J|$, and $\gamma \leftarrow \delta^2/(2 \log(2nm + 2))$

foreach $i \in N$ with $V_i(R) > 0$ **do**

 Following the order of R , ask agent i for a prefix worth $k\gamma V_i(R)$ for every positive integer k with $k\gamma < 1$, and record its endpoint

 After each cut, record every agent's value for the two resulting interval components as in Proposition C.1

Together, these cut points and the endpoints of the intervals forming R divide R into cells. Let $\mathcal{K}$ be their collection, and use the recorded values to compute how every agent values every cell

foreach labeling τ that assigns each cell in $\mathcal{K}$ an index in J or leaves it unused **do**

 Set $C_j(\tau) \leftarrow \bigcup \{K \in \mathcal{K} : \tau(K) = j\}$ for every $j \in J$

if $|V_i(C_j(\tau)) - \lambda_j V_i(R)| \leq \delta V_i(R)$ for every $i \in N$ and $j \in J$ **then**

return $(C_j(\tau))_{j \in J}$
